

Reasoning Runtime Behavior of a Program with LLM: How Far Are We?

Junkai Chen*
School of Software Technology,
Zhejiang University
Ningbo, China
junkaichen@zju.edu.cn

Zhiyuan Pan*
The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
zy_pan@zju.edu.cn

Xing Hu†
The State Key Laboratory of
Blockchain and Data Security,
Zhejiang University
Hangzhou, China
xinghu@zju.edu.cn

Zhenhao Li
York University
Toronto, Canada
lzhenhao@yorku.ca

Ge Li
Peking University
Beijing, China
lige@pku.edu.cn

Xin Xia
Zhejiang University
Hangzhou, China
xin.xia@acm.org

Abstract—Large language models for code (i.e., code LLMs) have shown strong code understanding and generation capabilities. To evaluate the capabilities of code LLMs in various aspects, many benchmarks have been proposed (e.g., HumanEval and ClassEval). Code reasoning is one of the most essential abilities of code LLMs (i.e., predicting code execution behaviors such as program output and execution path), but existing benchmarks for code reasoning are not sufficient. Typically, they focus on predicting the input and output of a program, ignoring the evaluation of the intermediate behavior during program execution, as well as the logical consistency (e.g., the model should not give the correct output if the prediction of execution path is wrong) when performing the reasoning. To address these problems, in this paper, we propose a framework, namely $\mathcal{REval}$, for evaluating code reasoning abilities and consistency of code LLMs with program execution. We utilize existing code benchmarks and adapt them to new benchmarks within our framework. A large-scale empirical study is conducted and most LLMs show unsatisfactory performance on both Runtime Behavior Reasoning (i.e., an average accuracy of 44.4%) and Incremental Consistency Evaluation (i.e., an average IC score of 10.3). Evaluation results of current code LLMs reflect the urgent need for the community to strengthen the code reasoning capability of code LLMs. Our code, data and $\mathcal{REval}$ leaderboard are available at <https://r-eval.github.io>.

Index Terms—Code Reasoning, Large Language Model, Benchmark

I. INTRODUCTION

Large language models (LLMs) attract great attention for their exceptional performance on diverse tasks [1] including sentiment analysis [2], logical reasoning [3], and question answering [4]. Recently, large language models for code (i.e., code LLMs) have become a popular research area because of the promising prospect of empowering humans in software development and maintenance [5]. Hence, both academia and industry have proposed a lot of code LLMs (e.g., CodeLlama

family [6] and Magicoder series [7]), which are widely applied to different tasks like code generation [8], [9].

To provide a fair and comprehensive measure of the capabilities of code LLMs, many code-related benchmarks (e.g., HumanEval [8] and CodeXGLUE [10]) are proposed to evaluate the effectiveness of code LLMs in different tasks such as code generation and vulnerability detection [11]. Given that “executable” is a distinct feature of code compared to natural language, and code execution provides additional information (e.g., program output) to assist with code tasks [12], [13], benchmarking code reasoning abilities of code models with execution raises researchers’ interests [14], [15]. Here, code reasoning is referred to as predicting code execution behaviors (e.g., program outputs, execution paths and possible variable values) without executing the code directly. For example, Gu et al. [14] proposed CRUXEval to evaluate code LLMs by predicting output from input and vice versa. Typically, these works measure the model’s ability to predict and analyze the relationship between the input and output of an executable program. However, the intermediate information (e.g., execution path) *during* code execution is ignored, posing challenges to developers in comprehending the program’s runtime behavior.

Fig. 1 shows common concerns about the *runtime behavior* during program execution. Intuitively, how a program behaves under certain input can help developers better understand the code and perform debugging activities. For example, if we have concerns about the correctness of a certain statement while debugging a snippet of code, we typically first determine whether this statement is executed given the input (i.e., ❶ in Fig. 1); If it is executed, observing the changes in variables before and after execution is a natural choice ❷; Sometimes this line of code may seem fine, so the statement immediately following it will be examined ❸; Additionally, the program output can be used to verify whether the results match the expectations ❹. Therefore, we argue that these kinds of runtime behaviors (e.g., program state and execution path) are

* Equal contribution.

† Corresponding author.

基于大语言模型的程序推理运行时行为：我们已经取得了多大进展？

陈俊凯 浙江大学软件技术
学院 中国宁波 junkaichen@
zju.edu.cn

李振豪
约克大学加拿大多伦多 lzh-
enhao@yorku.ca

潘志远* 浙江大学区块链与
数据安全国家重点实验室中
国杭州
zy_pan@zju.edu.cn

葛丽 北京大学中国北京 li-
ge@pku.edu.cn

薛欣† 浙江大学区块链与数
据安全国家重点实验室中国
杭州 xinghu@zju.edu.cn

新霞 浙江大学杭
州，中国 xin.xia@
acm.org

摘要——面向代码的大语言模型（即代码LLM）已展现出强大的代码理解与生成能力。为评估代码LLM在不同方面的性能，学界提出了多种评估基准（如HumanEval和ClassEval）。代码推理是代码LLM最重要的能力之一（即预测代码执行行为，如程序输出结果及执行路径），但现有的代码推理评估方法仍存在不足：这些方法通常仅关注预测程序的输入与输出结果，忽视了对程序执行过程中中间行为的评估以及逻辑一致性考量（例如当执行路径预测错误时，模型不应给出正确输出）。为解决这些问题，本文提出了一种名为REval的框架，用于评估代码LLM在代码推理能力和代码与程序执行一致性的方面表现。我们利用现有代码评估基准，并将其适配至本框架内的新评估指标中。我们开展了一项大规模实证研究，结果显示大多数代码大语言模型在运行时行为推理（平均准确率44.4%）和增量一致性评估（平均IC得分10.3）两个维度上均表现不尽如人意。现有代码大语言模型的评估结果表明，业界亟需提升其代码推理能力。我们的代码库、数据集及REval排行榜可访问<https://r-eval.github.io>获取。

索引/术语——代码推理、大型语言模型、基准测试

I. 引言

大型语言模型（LLMs）因其在多种任务[1]中表现出色而备受关注，这些任务包括情感分析[2]、逻辑推理[3]以及问答处理[4]。近年来，面向代码的大型语言模型（即代码LLMs）已成为热门研究领域，这得益于其在软件开发与维护过程中赋能人类的强大潜力[5]。因此，学术界和工业界都提出了众多代码LLMs（例如CodeLlama）。

Family系列[6]和Magicoder系列[7]，已被广泛应用于代码生成[8]等多种任务中。[9]。

为公平全面地评估代码大语言模型的能力，目前已提出多种与代码相关的基准测试方法（例如HumanEval[8]和CodeXGLUE[10]），用于检验代码大语言模型在代码生成、漏洞检测等不同任务中的表现[11]。鉴于“可执行性”是代码相较于自然语言的独特特征，且代码执行能提供额外信息（如程序输出）以辅助代码相关任务[12]，[13]通过将代码模型的推理能力与执行结果进行对比评估，激发了研究人员的兴趣[14]。[15]此处，代码推理指在不直接执行代码的情况下预测代码执行行为（例如程序输出结果、执行路径及可能的变量值）。例如，Gu等人。[14]我们提出了CRUXEval方法，通过预测输入与输出之间的关系来评估代码大语言模型。通常这类研究主要衡量模型预测和分析可执行程序输入与输出之间关系的能力。然而，代码执行过程中产生的中间信息（如执行路径）往往被忽视，这给开发者理解程序的运行时行为带来挑战。图1展示了关于程序执行期间运行时行为的常见关注点。直观而言，了解程序在特定输入下的行为表现有助于开发者更好地理解代码并进行调试操作。例如，在调试代码片段时若对某条语句的正确性存疑，我们通常首先需要验证该语句在给定输入条件下是否会被执行（即图中①所示情况）。①；若执行该代码段，则观察其执行前后的变量变化是自然的选择②；有时这段代码看似合理，因此会立即检查紧随其后的语句③；此外，程序输出结果可用于验证实际结果是否符合预期④。因此我们认为，这类运行时行为（如程序状态和执行路径）具有重要价值。

* 等额贡献†
通讯作者。

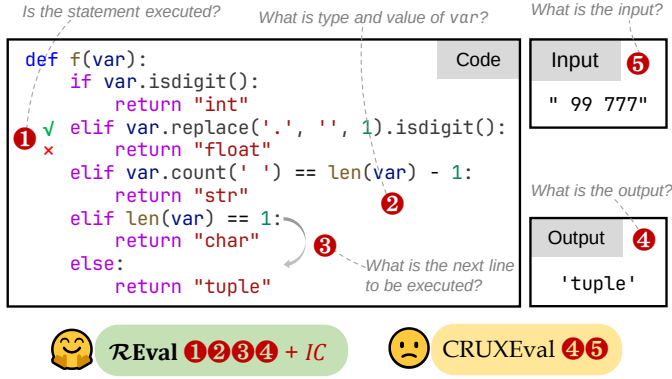


Fig. 1. The demonstration of code reasoning tasks in CRUXEval [14] and REval. “IC”: Incremental Consistency.

essential for program understanding and reasoning for humans. Meanwhile, they are also proven to be effective for an in-depth understanding of code semantics for language models [16]. As previous benchmarks like CRUXEval (i.e., with ④ and ⑤) fail to evaluate whether LLMs can reason about these dynamic characteristics of a program, it is necessary to measure the code reasoning ability of LLMs with runtime behavior of execution. In this paper, we propose our framework, **REval**, to comprehensively (re)-evaluate the code reasoning ability of LLMs, which consists of two evaluation components: (i) Runtime Behavior Reasoning and (ii) Incremental Consistency Evaluation.

Evaluation Component 1: Runtime Behavior Reasoning.

To mitigate this limitation in previous research, we make the first attempt to systematically evaluate the code LLM’s ability to reason about the runtime behavior of program execution. Specifically, we propose four evaluation tasks to achieve this goal: ① Code Coverage Prediction (CCP), i.e., whether a statement is executed or not; ② Program State Prediction (PSP), i.e., what is the value and the type of a variable; ③ Execution Path Prediction (EPP), i.e., which is the next statement to be executed; and ④ Output Prediction (OP), i.e., what is the output. These four tasks cover various aspects of program execution, including control flow, data flow, and type dependency, which are widely applied to prior research in software engineering such as type inference [17] and code translation [18]. Therefore, this evaluation provides a more comprehensive measure of code model’s ability to reason about executable programs in comparison with previous work.

Nevertheless, it is noticed that sometimes the reasoning results of a model could conflict with human logic on *sequential* tasks in Runtime Behavior Reasoning. For instance, the code model may correctly predict the next statement to be executed (i.e., EPP) when it fails to tell the value of a variable after the statement’s execution (i.e., PSP), which is not expected because the control flow of the execution relies on the program state. As this kind of *inconsistency in sequentially related tasks* is unlikely to occur in humans, the trustworthiness of AI systems built on these models (e.g., GitHub Copilot [19]) can easily suffer from these unreliable behaviors. Although

some previous works have discussed consistency for code LLMs [20], [21], they are limited to semantic consistency like back translation between NL and code and ignore the logical consistency mentioned here. Hence, it is necessary to measure the consistency of code LLMs on sequentially related tasks.

Evaluation Component 2: Incremental Consistency Evaluation.

To fill the gap in evaluation, we propose a novel metric named Incremental Consistency (IC) to measure the extent to which the model can maintain its logical consistency on sequentially related tasks of *incremental* difficulty. We observe that the four tasks in Runtime Behavior Reasoning are progressive and consistent with the context of IC, i.e., the knowledge required to finish the current task is the preliminary of the next task. Hence, we can judge how much a model is incrementally consistent by utilizing the results of reasoning runtime behavior (See Section III for details). Incremental Consistency provides new sights for evaluating LLMs and the consistency measure of AI systems beyond traditional metrics.

To construct our framework, we leverage existing executable datasets (e.g., HumanEval [8] and ClassEval [22]) as our base benchmarks and adapt them into an adapted benchmark within our framework by extracting runtime behavior, constructing, and filtering the problems. We conduct a large-scale empirical study on various models, including general and code LLMs in our frameworks. Evaluation results show that our framework presents a degree of difficulty and most LLMs show poor performance on both Runtime Behavior Reasoning and IC Evaluation (e.g., an IC score below 20 for all open-source LLMs we evaluate). Our research highlights the importance of utilizing runtime behavior and incremental consistency evaluation to measure the reasoning ability of code LLMs, and we call for targeted efforts in subsequent research to enhance these weaknesses.

In summary, the contributions of our paper are as follows:

- We propose a new framework, **REval**, to comprehensively evaluate code LLMs’ abilities of code reasoning. To the best of our knowledge, we are the first work to evaluate code models to systematically reason about runtime behavior during program execution.
- We propose a novel metric named Incremental Consistency (IC) to measure to what extent a code LLM can maintain its consistency across sequentially related tasks of incremental difficulty.
- We conduct a large-scale empirical study on diverse LLMs within our evaluation framework. Our results reveal the limitations of reasoning runtime behavior and IC of code models.
- We construct an adapted benchmark based on HumanEval [8] and ClassEval [22] and develop an evaluation harness for our framework. To facilitate further research of code reasoning, our code, data, and **REval** leaderboard are publicly available at <https://r-eval.github.io>.

II. BACKGROUND AND RELATED WORK

In this section, we discuss the background information of our research and the corresponding related work.

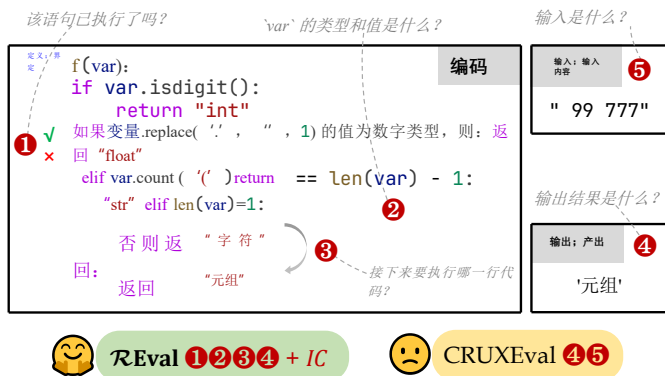


图 1. CRUXEval[14] 和 REval 中代码推理任务的演示。“IC”：增量一致性。

这对人类理解程序逻辑和进行推理至关重要。同时，它们也被证明能有效帮助语言模型深入理解代码语义[16]。由于以往的基准测试（如CRUXEval，即使用④和⑤）未能评估大语言模型是否能够对程序的这些动态特性进行推理，因此有必要通过执行时的行为表现来衡量大语言模型的代码推理能力。本文提出我们的框架REval，用于全面（re）评估大语言模型的代码推理能力，该框架包含两个评估组件：(i)运行时行为推理；(ii)增量一致性评估。

评估组件 1：运行时行为推理。

为缓解先前研究中的这一局限性，我们首次系统性地评估了代码大语言模型（LLM）对程序执行运行时行为进行推理的能力。具体而言，我们提出了四项评估任务以实现这一目标：①代码覆盖率预测（立方最密堆积），即判断某条语句是否被执行；②程序状态预测（PSP），即确定变量的值及其类型；③执行路径预测（EPP），即预测下一条将被执行的语句；以及④输出预测（OP），即预测输出结果。这四项任务涵盖了程序执行的多个方面，包括控制流、数据流和类型依赖性，这些内容在软件工程领域的现有研究中广泛应用，例如类型推断[17]和代码翻译[18]。因此，相较于以往研究，本评估方法能更全面地衡量代码模型对可执行程序推理能力。然而需要指出的是，在运行时行为推理的*顺序*任务中，模型的推理结果有时可能与人类逻辑存在冲突。例如，当代码模型无法准确预测语句执行后的变量值（即PSP）时——这种情况本就不应发生，因为程序执行的控制流程依赖于程序状态——它仍可能正确预测出待执行的下一个语句（即EPP）。由于人类在处理*顺序相关任务时几乎不会出现此类不一致性问题*，基于此类模型构建的人工智能系统（如GitHub Copilot[19]）的可靠性极易因这些不可靠行为而受到影响。尽管如此.....

先前的一些研究已探讨了代码型大语言模型的一致性[20]。[21]这些方法仅关注语义一致性（如自然语言与代码之间的反向翻译），而忽略了本文所提及的逻辑一致性。因此，有必要评估代码大语言模型在序列相关任务中的一致性表现。

评估组件2：增量一致性评估。为填补现有评估领域的空白，我们提出了一种名为“增量一致性（IC）”的新指标，用于衡量模型在具有*递增难度*的连续相关任务中保持逻辑一致性的程度。我们发现，“运行时行为推理”中的四个任务均具有渐进性且符合IC的语境要求——即完成当前任务所需的知识是下一任务的前置条件。因此，可通过推理运行时行为的结果来判断模型的增量一致性水平（详见第三节）。增量一致性为评估大语言模型及超越传统指标的人工智能系统一致性提供了新的视角。

为构建我们的框架，我们利用现有的可执行数据集（如HumanEval[8]和ClassEval[22]）作为基础基准，并通过提取运行时行为、构建并筛选问题实例，将其转化为适用于本框架的定制化评估指标。我们在包括通用模型和代码生成型大语言模型在内的多种模型上开展了大规模实证研究。评估结果表明：该框架存在一定难度阈值，且大多数大语言模型在运行时行为推理和集成一致性评估方面表现欠佳（例如所有被评估开源大语言模型的集成一致性得分均低于20）。我们的研究凸显了利用运行时行为特征与增量一致性评估来衡量代码生成型大语言模型推理能力的重要性，并呼吁后续研究应针对性地改进这些不足之处。

综上所述，本文的主要贡献如下：

- 我们提出了一种新的评估框架REval，用于全面评估代码大语言模型的代码推理能力。据我们所知，这是首个系统性评估代码模型在程序执行过程中对运行时行为进行推理的研究工作。
- 我们提出了一种名为“增量一致性”的新型度量指标。（IC）用于评估代码大型语言模型（LLM）在递增难度的连续相关任务中保持一致性的程度。
- 我们在评估框架内对多种大语言模型（LLM）开展了大规模实证研究。研究结果揭示了代码模型推理运行时行为及信息闭包性（IC）存在的局限性。
- 我们基于HumanEval[8]和ClassEval[22]构建了一个定制化基准，并为我们的框架开发了评估工具包。为促进代码推理领域的进一步研究，我们的代码库、数据集以及REval排行榜已公开发布于<https://r-eval.github.io>。

II. 背景及相关工作

在本节中，我们阐述了本研究的背景信息及相关已有研究成果。

A. Code Execution and Reasoning

1) *Code Execution Behavior*: We refer to code execution behavior as the additional information offered by program execution compared to static analysis. According to the execution order, we classify them into pre/post-execution information and runtime information:

- *Pre/Post-Execution Information* is the content we can obtain before or after the actual execution process of program. For example, the input, output, and NL requirements belong to this category.
- *Runtime Information* is the intermediate state during code execution. For instance, we are able to collect contents like program state and execution path only when the code is still running.

Previous research has leveraged code execution behavior to improve the performance of various downstream tasks, e.g., program understanding [23], code generation [12], [13], [24], software testing [25], [26] and vulnerability detection [16]. Ni et al. [12] improved code generation performance with an extra verifier, which learns the results of code execution and helps rerank generated code candidates. Chen et al. [13] utilized different kinds of feedback including output to help LLMs “self-debug” the generated code. They designed a series of prompting strategies to guide the model to refine the program automatically. In these works, pre-/post-execution information, such as program output, is applied to code generation. Furthermore, some works found the worth of dynamic features during execution and exploit them to train various language models. Liu et al. [24] pre-trained a language model to learn the execution process of the program. Specifically, they represented the program state as a sequence that neural models can learn from and expect the model to predict the trace. Compared to Liu et al., Ding et al. [16] proposed a pre-training technique combining both static and dynamic characteristics of the program. In summary, the aforementioned works reflect the close relationship between the behavior of code execution and the program semantics, and emphasize the importance of evaluating models for code reasoning with execution.

2) *Code Reasoning with Large Language Models*: As introduced in Section I, in the task of code reasoning, an LLM needs to predict the program behavior without execution.

Recently, some works have proposed different evaluation approaches for the code reasoning abilities of code LLMs. For example, Gu et al. [14] proposed CRUXEval, which requires LLMs to reason about pre/post-execution information such as input and output. Following this study, similar to the idea of CRUXEval, Liu et al. [15] extended the evaluation tasks (i.e., predicting input and output) to the natural language specification. However, their evaluation approaches are still limited to pre/post-execution information and ignore intermediate runtime behavior. In contrast, our work goes a step further to measure how the model learns the runtime behavior *during* execution, which shows promising potential in helping program comprehension, as mentioned above. **We notice that a recent work [27] aimed to simulate the code execution**

process with code LLMs. They used the analogy of a large language model to a CPU to explore the process of a program executing code, paying more attention to algorithm complexity and structure. Different from the aforementioned studies, our framework is not only limited to algorithm problems (e.g., competition-level ones), but also suitable for general programming scenarios (e.g., more real-world projects). In addition, we also explore detailed runtime behavior like code coverage and execution path, containing more runtime information.

B. Consistency for Large Language Models

Semantic Consistency. Semantic consistency refers to the same decisions on semantically equivalent texts of LLMs [28]. For example, the model should provide similar and even the same answers in the face of two meaning-perserving questions. In the realm of software engineering, this feature is generally utilized for the unsupervised evaluation of code LLMs [20], [21]: Min et al. [21] evaluated the self-consistency of code LLMs by comparing the functional correctness of two code snippets: one code is generated using a human-written description, and the other is generated iteratively with the summary of the previously generated. **Chen et al. [29] studied the robustness of code LLMs to the variations in natural language descriptions for code generation.** Allamanis et al. [20] introduced round-trip correctness which aligns code and NL to perform unsupervised evaluation for code LLMs. The aforementioned works leveraged the back translation between NL and PL iteratively generated by the model and conduct the semantic or functional comparison. However, they are restricted to the context of semantic consistency in the context of NL and PL.

Logical Consistency. **If an LLM is able to make predictions without logical contradiction, it shows the feature of logical consistency [28].** For example, if one model assumes a proposition to be true, it should consider the negation of that proposition to be false as well. There are lots of previous research about how to evaluate and utilize logical consistency for LLMs in natural language processing [28], [30]–[32], **but few works pay attention to logical consistency on code LLMs.** As the reasoning ability is highly related to its logical consistency [33], a comprehensive code reasoning evaluation should contains the measure of logical consistency in scope of programming languages (PLs). Therefore, it motivates us to propose a novel consistency metric idea named IC to fill this gap.

C. Code LLMs and Benchmarks

1) *Code LLMs*: Large language models for code are LLMs specialized for the generation and understanding of PLs. For example, CodeLlama family models [6] inherit the architecture of Llama2 [34] and are further pre-trained on extra code corpora. Its three variant models (i.e., *base*, *instruct*, and *Python-specialized*) are designed for different programming scenarios. StarCoder2 [35] is a series of code LLMs developed by the BigCode Project, which achieves competitive performance with other similar-sized models. These models are trained

A. 代码执行与推理

1) **代码执行行为**: 我们指的是代码的执行过程行为信息是程序执行相较于静态分析所提供的额外信息。根据执行顺序, 可将其分为执行前/后信息和运行时信息:

- **执行前/执行后信息** 是我们在程序实际执行过程之前或之后能够获取的内容。例如, 输入、输出及自然语言处理需求均属于此类信息。
- **运行时信息** 是代码执行过程中的中间状态。例如, 只有在代码仍在运行时, 我们才能收集程序状态和执行路径等信息。

先前的研究利用代码执行行为来提升各类下游任务的性能, 例如程序理解[23]、代码生成[12]。[13], [24]软件测试[25][26] 以及漏洞检测[16]。Ni等人。[12] 通过增加一个额外的验证器, 代码生成性能得到提升; 该验证器可学习代码执行结果, 并协助对生成的代码候选方案进行重新排序。Chen等人[13] 研究团队采用了多种反馈方式(包括输出结果)来帮助大语言模型对生成的代码进行“自我调试”。他们设计了一系列提示策略, 引导模型自动优化程序代码。这些研究中, 执行前/后的信息(如程序输出结果)均被应用于代码生成过程; 此外, 部分研究还发现了运行过程中动态特征的价值, 并利用这些特征训练各类语言模型。Liu等人亦开展了相关研究。[24] 对语言模型进行预训练, 使其能够学习程序的执行过程。具体而言, 他们将程序状态表示为神经网络模型可学习的序列, 并期望模型据此预测执行轨迹。与Liu等人的研究相比, Ding等人的方法更具优势。[16] 提出了一种结合程序静态与动态特性的预训练技术。总之, 上述研究充分体现了代码执行行为与程序语义之间的密切关联, 并强调了在实际执行过程中评估代码推理模型的重要性。

2) 基于大型语言模型的代码推理: 如同

如第I节所述, 在代码推理任务中, 大型语言模型(LLM)需要在不执行程序的情况下预测其行为。

近期, 多项研究提出了评估代码大语言模型(LLM)代码推理能力的不同方法。例如, Gu等人提出了一种方法。[14] 提出的CRUXEval方法要求大型语言模型(LLM)对执行前/后信息(如输入和输出)进行推理。基于这项研究, Liu等人提出了与CRUXEval理念相似的方案。[15] 将评估任务(即预测输入和输出)扩展到了自然语言规范领域。然而, 他们的评估方法仍局限于执行前/后的信息, 且忽略了中间运行时的行为。相比之下, 我们的研究更进一步地测量了模型在**执行过程中**如何学习运行时行为, 正如上文所述, 这在促进程序理解方面展现出巨大潜力。我们注意到近期一项研究[27]旨在模拟代码执行过程。

我们采用代码大语言模型(LLM)进行研究。通过将大型语言模型比作CPU来分析程序执行代码的过程, 重点探讨算法的复杂性与结构特征。与前述研究不同, 我们的框架不仅适用于算法问题(如竞赛级别的难题), 还广泛适用于各类编程场景(如实际应用项目)。此外, 我们还深入探究了运行时的详细行为特征, 包括代码覆盖率和执行路径等更丰富的运行时信息。

B. 大型语言模型的一致性

语义一致性。语义一致性指的是大型语言模型对语义等效文本作出相同的决策[28]。例如, 面对两个保持语义一致性的问题时, 模型应给出相似甚至完全相同的答案。在软件工程领域, 这一特性通常被用于代码生成型大语言模型的无监督评估[20]。[21]米恩等人。[21] 通过比较两个代码片段的功能正确性, 评估了代码大语言模型(LLM)的自洽性: 其中一个代码基于人工编写描述生成, 另一个则根据先前生成结果的摘要进行迭代生成。Chen等人。[29] 研究了代码大语言模型(LLM)在代码生成过程中对自然语言描述变异的鲁棒性。Allamanis等人。[20] 本文引入了往返正确性机制, 该机制可同步代码与自然语言(NL), 从而对代码大语言模型(LLM)进行无监督评估。上述研究均利用模型迭代生成的自然语言与编程语言之间的反向翻译结果, 并开展语义或功能层面的比较; 但这些方法仅适用于自然语言与编程语言之间的语义一致性分析场景。

逻辑一致性。如果一个大语言模型能够在预测时避免逻辑矛盾, 便体现了其逻辑一致性特征[28]。例如, 若某个模型认为某个命题为真, 则应将该命题的否定结果视为假。关于如何在自然语言处理领域评估并利用大语言模型的逻辑一致性, 已有大量相关研究[28]。[30]–[32], 但鲜有研究关注代码大语言模型的逻辑一致性问题。由于推理能力与其逻辑一致性密切相关[33], 全面的代码推理评估应包含编程语言(PLs)层面的逻辑一致性度量指标。因此, 我们提出一种名为IC的新一致性度量指标以填补这一空白。

C. 代码、LLM模型及基准测试

1) **代码大语言模型**: 专为代码生成与理解设计的大语言模型即针对编程语言生成与理解任务优化的LLM。例如, CodeLlama系列模型[6]继承了Llama2[34]的架构, 并在额外的代码语料库上进行了进一步预训练; 其三个变体模型(即**基础版**、**指令编写版**和**Python专用版**)分别针对不同的编程场景设计。StarCoder2[35]是由BigCode项目开发的一系列代码LLM模型, 其性能可与其他规模相近的模型相媲美。这些模型均经过专门训练

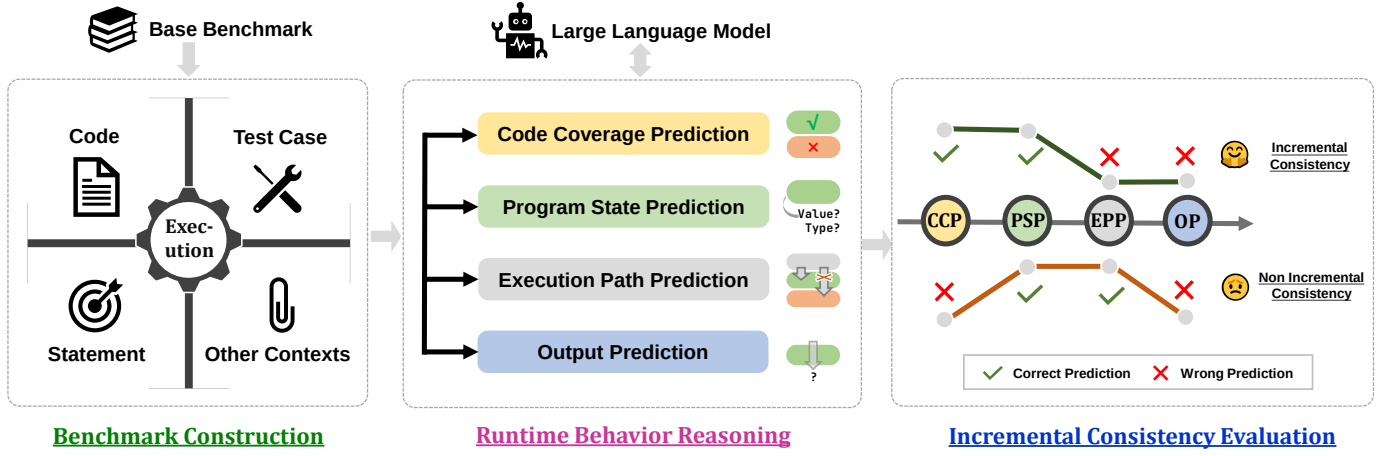


Fig. 2. Overview of our framework. **Benchmark Construction:** we adapt the base benchmarks to fit our framework by execution. **Runtime Behavior Reasoning:** we propose four tasks including CCP, PSP, EPP, and OP, which challenge LLMs to perform code reasoning. **Incremental Consistency Evaluation:** we evaluate if the model can maintain consistency on sequentially related tasks (i.e., Incremental Consistency).

on The Stack v2 dataset [35] whose data size is four times larger than its first generation. CodeGen2.5 models [36] are improved versions of their previous models (e.g., CodeGen2) for program synthesis. It is claimed that their performance gains mainly come from optimizations such as training and sampling strategies.

Code reasoning is one of the most important capabilities of LLMs related to code [14], [15], but few work are dedicated to its evaluation. In this paper, we aim at comprehensively evaluating their reasoning capabilities for programming languages based on execution.

2) *Benchmarks:* Recently, many code generation benchmarks have been proposed to evaluate the correctness of code snippets generated by code LLMs. HumanEval [8] is one of the most popular benchmarks for code generation. It consists of 164 competitive programming problems and evaluates the functional correctness of generated samples rather than text similarity. Apart from competitive benchmarks of which question generally runs in a simple context, lots of context-aware benchmarks such as CoderEval [37], ClassEval [22], and CrossCodeEval [38] have been proposed. These benchmarks provide more complex surrounding contexts and dependencies (e.g., private libraries) to evaluate code generation in real-world projects. In addition, some domain-specific benchmarks have been proposed to evaluate the performance of code generation in various programming languages and paradigms (e.g., DS-1000 [39] for data science). Our work utilizes existing executable benchmarks for code generation and evaluates code LLMs with respect to code reasoning, which makes our framework universal and applicable to different scenarios.

Prior studies proposed a variety of code understanding tasks including code search, type inference, and code translation. Evaluating the ability of code understanding is essential for code LLMs. Lu et al. [10] propose CodeXGLUE, a comprehensive benchmark for code models that supports 10 tasks related to code and text. Cassano et al. [40] create a hand-crafted

benchmark to evaluate the instruction following ability on code editing. Khan et al. [41] introduce a large-scale multilingual multitask benchmark that consists of numerous executable coding examples. Compared to the above benchmarks for code understanding, we propose a comprehensive framework for code reasoning from the perspective of runtime behavior, which provides a different point of view for the evaluation of code LLMs.

III. REVAL FRAMEWORK

In this section, we first introduce the overview of our evaluation framework **REval**, and then describe the two evaluation components in detail, namely, **Runtime Behavior Reasoning** and **Incremental Consistency Evaluation**. In the end, we describe how to construct the corresponding benchmark under our framework.

A. Overview of Framework

Fig. 2 shows an overview of our framework, which aims to challenge code LLMs to reason how the program behaves during execution. To achieve this, we adopt two different perspectives for the abilities of code reasoning: (1) *Runtime Behavior Reasoning*; and (2) *Incremental Consistency Evaluation*.

For Runtime Behavior Reasoning, we focus on whether the code model can correctly predict the intermediate states of program execution given an executable program and input (as well as other contexts in the base benchmark). We select four different dimensions of runtime behavior, including *code coverage*, *execution path*, *program state*, and *output*, each of which corresponds to a specific sub-task under Runtime Behavior Reasoning. We present the task description and the evaluation metric in Section III-B for each sub-task.

Besides the standalone metrics that measure a single capability of the models, we propose a novel idea named Incremental Consistency Evaluation to **assess the consistency across a series of incremental tasks during code reasoning**. As

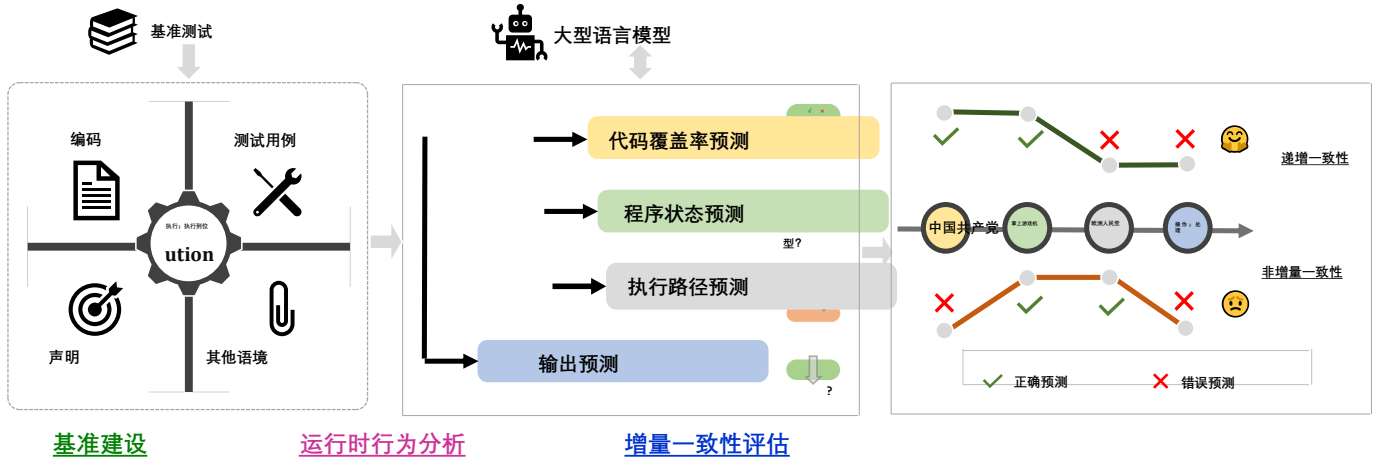


图2. 我们框架的概述。**基准测试构建**：我们根据具体执行需求调整基础基准测试以适配我们的框架。**运行时行为推理**：我们提出了包括立方最密堆积、PSP、EPP和OP在内的四项任务，旨在挑战大语言模型进行代码推理能力。**增量一致性评估**：我们评估模型是否能在序列相关任务中保持一致性（即增量一致性）。

在On The Stack v2数据集[35]上，其数据量是第一代数据集的四倍。CodeGen2.5模型[36]是针对程序合成设计的前代模型（如CodeGen2）的改进版本。据称其性能提升主要源于训练策略和采样策略等方面的优化。

代码推理是大语言模型在代码相关领域最重要的能力之一[14]。[15]然而，**目前鲜有研究专门针对其评估展开。本文旨在基于执行结果，全面评估编程语言的推理能力。**

2) 基准测试：近期，许多代码生成基准测试

已有多种评估指标被提出用于检验代码大语言模型生成的代码片段是否正确。HumanEval[8]是代码生成领域最流行的基准测试之一，它包含164道竞赛级编程问题，主要评估生成代码样本的功能正确性而非文本相似度。除这类通常在简单上下文中运行的问题外，还提出了许多具备上下文感知能力的基准测试工具，如CoderEval[37]、ClassEval[22]和CrossCodeEval[38]。这些基准测试通过提供更复杂的环境上下文与依赖关系（例如私有库），来评估实际项目中的代码生成效果。此外，还开发了针对特定领域的基准测试工具以评估不同编程语言及开发范式下的代码生成性能（例如面向数据科学的DS-1000[39]）。我们的研究利用现有的可执行代码生成基准测试工具，从代码推理角度对代码大语言模型进行评估，从而使该框架具有通用性并适用于多种场景。

先前的研究提出了多种代码理解任务，包括代码搜索、类型推断和代码翻译。评估代码理解能力对于代码大语言模型（LLM）至关重要。Lu等人。[10] 提出CodeXGLUE，这是一个针对代码模型的综合性基准测试工具，支持涵盖代码与文本相关的10项任务。Cassano等人。[40] 手工制作的.....

用于评估代码编辑中指令执行能力的基准指标。**Khan等人。[41]** 我们提出了一种大规模多语言多任务基准测试框架，其中包含大量可执行的代码示例。与上述代码理解基准相比，我们从运行时行为的角度提出了一个全面的代码推理框架，为代码大语言模型（LLM）的评估提供了全新的视角。

III. 重组框架

在本节中，我们首先介绍评估框架**Reval**的总体架构，随后详细阐述两个评估组件——即**运行时行为推理**与**增量一致性评估**；最后说明如何在该框架下构建相应的基准测试。

A. 框架概述

图2展示了我们框架的总体架构，该框架旨在挑战代码大语言模型（LLM）推理程序在执行过程中的行为方式。为实现这一目标，我们针对代码推理能力采用了两种不同视角：(1)**运行时行为推理**；(2)**增量一致性评估**。

在运行时行为推理方面，**我们重点研究代码模型能否根据可执行程序及其输入（以及基准测试中的其他上下文信息）准确预测程序执行过程中的中间状态。**我们选取了四个不同的运行时行为维度：**代码覆盖率**、**执行路径**、**程序状态**和**输出结果**，每个维度均对应运行时行为推理框架下的特定子任务。针对每个子任务的任务描述及评估指标详见III-B节。

除了用于衡量模型单一能力的独立指标外，我们提出了一种名为“增量一致性评估”的新方法，**用于在代码推理过程中评估一系列增量任务之间的一致性。**

the knowledge required to finish the latter task contains that of the former task in Runtime Behavior Reasoning, the difficulty increases progressively in order, and we can utilize this characteristic to evaluate Incremental Consistency of LLMs with existing predictions (see Section III-C for details).

In addition, as our evaluation relies on existing base benchmarks, we present how to construct an adapted benchmark of code reasoning within our framework in Section III-D.

B. Runtime Behavior Reasoning

Runtime behavior refers to the intermediate state and information during program execution, such as code coverage and variable values, which are widely mentioned in previous research [16], [42]. As shown in Fig. 2, to evaluate the reasoning ability in program runtime behavior for code models, we analyze and select four representative dimensions of intermediate information during execution, including *code coverage*, *execution path*, *program state*, and *output prediction*. Corresponding to these features, we propose four sub-tasks for Runtime Behavior Reasoning and introduce them in detail.

1) *Code Coverage Prediction (CCP)*: Code coverage measures the proportion of code covered by a test suite [43]. Recent research [42] utilized this idea to challenge the model in predicting whether the statements in the program can be executed or not. Hence, we exploit the LLM code to judge whether a specific statement is executed given the input of a test case.

Task Description. Given a program $\mathbf{P}$ with statements $(S_1, S_2, \dots, S_n)$, an input $\mathbf{X}$ for execution and a statement index I , the model $\mathcal{M}$ is required to predict whether the I -th statement S_I is executed. The ground truth can be denoted as $\text{Coverage}(I)$.

Evaluation Metrics. In this task, **Accuracy** presents the percentage of correct coverage predictions. For our benchmark that consists of N number of $(\mathbf{P}, \mathbf{X}, I)$ pairs, *Accuracy* can be computed as:

$$\text{Accuracy} = \frac{1}{N} \sum \left[\mathcal{M}(\mathbf{P}, \mathbf{X}, I) = \text{Coverage}(I) \right]$$

Besides, as this task can be considered as a binary classification task, we also use **F1 score** as the evaluation metric following previous research [24].

2) *Program State Prediction (PSP)*: The initial idea of *program state* refers to values of the program counter and the variables [44] in the context of assembly language and instructions. Since we are mainly concerned with code models at the source code level, we follow related work [16], [45] and define *program state* as a set of variables in the current runtime scope. Each variable has its corresponding value and type. Program State Prediction examines the model's ability to reason about value and type conversion of the variable after a statement is executed.

Task Description. Given a program $\mathbf{P}$ with statements $(S_1, S_2, \dots, S_n)$, an input $\mathbf{X}$, a statement index I and a

variable name V related to the current statement S_i , the model $\mathcal{M}$ is required to predict the type and value of variable V after S_I is executed. The ground truth of type and value can be denoted as $\text{Ty}(I, V)$ and $\text{Val}(I, V)$, respectively.

Evaluation Metrics. In this task, **Accuracy** (Acc.) measures the percentage of correct value and type predictions:

$$\text{Acc.} = \frac{1}{N} \sum \left[\mathcal{M}(\mathbf{P}, \mathbf{X}, I, V) = (\text{Val}(I, V), \text{Ty}(I, V)) \right]$$

With this equation, a model's prediction is correct only if the value and type both match the ground truth.

3) *Execution Path Prediction (EPP)*: In this task, we refer to the execution path of the program as ordered sequences of statements. As the granularity of our context is statement-level, we challenge the code model to predict the next statement to be executed given a specific statement. A code LLM skilled in code reasoning should be capable of telling where the control flow of the program is going and, consequently, can predict the next executed statement naturally.

Task Description. Given a program $\mathbf{P}$ with statements $(S_1, S_2, \dots, S_n)$, an input $\mathbf{X}$ and a statement index I , the model $\mathcal{M}$ is required to predict the next statement to be executed after S_I is executed. The ground truth can be denoted as $\text{Next}(I)$.

Evaluation Metrics. In this task, **Accuracy** measures the percentage of correct next statement predictions:

$$\text{Accuracy} = \frac{1}{N} \sum \left[\mathcal{M}(\mathbf{P}, \mathbf{X}, I) = \text{Next}(I) \right]$$

Note that if the number of possible answers is more than one (i.e., several statements could be the next one to execute), we consider the prediction correct if it hits any possible one.

4) *Output Prediction (OP)*: This task is to directly generate the output of a program with the given input, which is applied in previous code reasoning work [14], [15]. To accurately predict the output of a program, code LLMs should be capable of controlling and simulating the whole execution process, which places high demands on the code reasoning ability. To evaluate the correctness of the output, we utilize test cases (i.e., a collection of assertion statements) in the base benchmarks. This approach is also applied in various code benchmarks [8], [22].

Task Description. Given a program $\mathbf{P}$ and an input $\mathbf{X}$ for execution, the model $\mathcal{M}$ is required to generate the output. The correct output can be denoted as $\mathbf{Y}$.

Evaluation Metrics. In this task, **Accuracy** measures the percentage of correct output predictions:

$$\text{Accuracy} = \frac{1}{N'} \sum \left[\mathcal{M}(\mathbf{P}, \mathbf{X}) = \mathbf{Y} \right],$$

where N' equals the number of different $(\mathbf{P}, \mathbf{X})$ pairs in our benchmark.

完成后一任务所需的知识包含了前一任务在运行时行为推理方面的知识，其难度按顺序逐步递增；我们可以利用这一特性来评估大语言模型与现有预测结果之间的增量一致性（详见III-C节）。

此外，由于我们的评估基于现有的基础基准测试，我们在第III-D节中阐述了如何在我们的框架内构建适用于代码推理的定制化基准测试。

B. 运行时行为分析

运行时行为指程序执行过程中的中间状态及相关信息（如代码覆盖率和变量值），这些内容在先前研究[16]中已被广泛提及。[42]如图2所示，为评估代码模型在程序运行时行为中的推理能力，我们分析并选取了执行过程中四个具有代表性的中间信息维度，包括代码覆盖率、执行路径、程序状态以及输出预测。基于这些特征，我们提出了四项运行时行为推理子任务并进行了详细阐述。

1) **代码覆盖率预测（立方最密堆积）**：代码覆盖率衡量测试套件所覆盖的代码比例[43]。近期研究[42]利用这一概念来检验模型在预测程序中语句是否可执行方面的性能。因此，我们利用大语言模型代码来判断给定测试用例输入时特定语句是否会被执行。

任务描述。给定一个包含语句 $(S_1, S_2, \dots, S_n)$ 的程序 P 、用于执行的输入 X 以及语句索引 I ，模型 M 需要预测第 I 条语句 S_I 是否会被执行。真实值可表示为覆盖率 (I) 。

评估指标。在此任务中，**准确率**表示正确覆盖预测的百分比。作为我们的基准测试该集合由 N 个 (P, X, I) 对组成，**准确率**可计算如下：

$$\text{准确率} = \frac{1}{N} \sum \mathbb{I}(\text{条件表达式. 值为0或1.}) \text{JM}(P, X, I) = \text{覆盖率}(I)K$$

此外，由于该任务可视为二分类任务，我们还参照先前研究[24]，采用**F1分数**作为评估指标。

2) **程序状态预测（PSP）**：程序状态的初始概念指的是汇编语言及指令语境下的程序计数器值与变量值[44]。由于我们主要关注源代码层面的代码模型，因此参考了相关研究[16]。[45]并将**程序状态**定义为当前运行时作用域中的一组变量。每个变量均具有相应的值和类型。程序状态预测用于评估模型在语句执行后对变量的值转换和类型转换进行推理的能力。

任务描述。给定一个包含语句 $(S_1, S_2, \dots, S_n)$ 的程序 P 、输入 X 、语句索引 I 以及一个

变量名称 V 与当前语句 S_I 相关，模型 M 需要在执行 S_I 后预测变量 V 的类型和值。类型与值的真实值分别表示为 $Ty(I, V)$ 和 $Val(I, V)$ 。**评估指标**：在此任务中，**准确率**（Acc.）衡量正确值预测与类型预测的比例：

$$\text{Acc.} = \frac{1}{N} \sum \mathbb{I}(\text{JM}(P, X, I, V) = (Val(I, V), Ty(I, V)))K$$

利用该方程，模型的预测仅在满足特定条件时才正确。其数值和类型均与真实值一致。

3) **执行路径预测（EPP）**：在此任务中，我们将程序的执行路径定义为语句的有序序列。由于我们的上下文粒度为语句级别，该任务旨在要求**代码模型根据给定特定语句预测后续将被执行的语句**。具备代码推理能力的代码大语言模型应能够判断程序控制流的走向，并据此进行预测。接下来的执行语句自然随之而来。

任务描述。给定一个包含语句 $(S_1, S_2, \dots, S_n)$ 的程序 P 、输入 X 以及语句索引 I ，模型 M 需要预测在执行到 S_I 之后将要执行的下一个语句。真实值可表示为 $Next(I)$ 。

评估指标。在此任务中，**准确率**表示正确预测后续陈述的百分比：

$$\text{准确率} = \frac{1}{N} \sum \mathbb{I}(\text{JM}(P, X, I) = Next(I))K$$

需注意，若可能答案的数量超过一个（即可能存在多个后续可执行的语句），则只要预测结果符合任意一种可能情况，即视为正确。

4) **输出预测（OP）**：该任务旨在根据**给定输入直接生成程序的输出结果**，该方法已应用于先前的代码推理研究[14]。[15]要准确预测程序的输出结果，代码大语言模型（LLM）必须能够控制并模拟整个执行过程，这对代码的推理能力提出了极高要求。为评估输出结果的正确性，我们采用基准测试中的测试用例（即一组断言语句）。该方法亦被应用于多种代码基准测试中[8]、[22]。

任务描述。给定一个程序 P 和用于执行的输入 X ，模型 M 需要生成输出结果。正确的输出可表示为 Y 。

评估指标。在此任务中，**准确率**表示正确输出预测结果的百分比：

$$\text{准确率} = \frac{1}{N'} \sum \mathbb{I}(\text{JM}(P, X) = Y)K,$$

其中 N' 表示我们基准测试中不同 (P, X) 组合的数量。

C. Incremental Consistency Evaluation

Incremental Consistency refers to the idea of how much the model can maintain its consistency across a series of sequentially related tasks. Intuitively, if an LLM cannot reason about the current task, it is not expected to finish the next task whose preliminary depends on the current task. To clarify this idea, we first present the description of Incremental Consistency, and then explain how we evaluate it on code models in practice.

1) *Description of Incremental Consistency*: The core idea of Incremental Consistency is to assess code models by leveraging the relationship where the knowledge from one task in a series of tasks depends on the next task. In the context of our research, four distinct sub-tasks (i.e., CCP, PSP, EPP, and OP) in Runtime Behavior Reasoning are selected, and we can observe some patterns from them:

- i) $\text{CPP} \Leftarrow \text{PSP}$: Since the execution of a statement could lead to changes in the program state, the prerequisite for PSP is correctly predicting if the statement is executed (i.e., CCP).
- ii) $\text{PSP} \Leftarrow \text{EPP}$: The control flow of a running program is affected by the value of some variables (e.g., “if” branch and its conditional variable), thus the next statement to be executed (i.e., EPP) is influenced by the program state (i.e., PSP).
- iii) $\text{EPP} \Leftarrow \text{OP}$: The intermediate execution state is one of the factors that affect the program output. Thus, the knowledge for OP covers that of PSP.

According to the above descriptions, we find that the knowledge required to finish the previous task is contained by that of the following task. Intuitively, the subsequent tasks are more difficult than the current task. Hence, if a model fails to correctly complete a task (e.g., fails to finish CPP) but then predicts the following tasks correctly (e.g., correct prediction of output), we consider this model behaves inconsistently in consecutive tasks.

2) *Evaluation Approach*: We analyze the results of our Runtime Behavior Reasoning to evaluate the Incremental Consistency. Specifically, for the i -th specific problem in our benchmark (i.e., full program, a specific statement in it, input, and one question to ask), we assume that the sequential results of four tasks are:

$$\mathbf{R}_i = \{r_{\text{CPP}}, r_{\text{PSP}}, r_{\text{EPP}}, r_{\text{OP}}\},$$

where $r \in \{0, 1\}$ and the number indicates whether the prediction matches the ground truth (i.e., 1) or not (i.e., 0). Hence, if the model is incrementally consistent and completes one task only when all its previous tasks are finished, the binary sequence $\mathbf{R}_i$ should be non-declining, i.e.,

$$\mathbf{R}_i \in \mathbf{S}, \quad \text{where } \mathbf{S} = \{\{1, 1, 1, 1\}, \{1, 1, 1, 0\}, \{1, 1, 0, 0\}, \{1, 0, 0, 0\}\}.$$

For the first example of Incremental Consistency Evaluation in Fig. 2, the resulting sequence is $\{1, 1, 0, 0\}$ (✓, ✓, ✗, ✗), which means that Incremental Consistency is observed in this

case. However, for the second example, the result is $\{0, 1, 1, 0\}$ (✗, ✓, ✓, ✗), so Incremental consistency is not observed.

In addition, depending on how many consecutive times consistency is maintained (i.e. $\{1, 1, 1, 1\}$ means 4 times), we assign different weights to reward models that maintain Incremental Consistency more often. It is intuitive because it is harder to behave incrementally consistently across more sub-tasks.

Finally, we define **Incremental Consistency Score** (IC Score) to quantitatively model the Incremental Consistency of an LLM $\mathcal{M}$. For our benchmark that contains N number of $(\mathbf{P}, \mathbf{X}, I, V)$ pairs, IC Score can be computed as:

$$\text{IC Score} = \frac{100}{N} \sum_{i=1}^N \text{IC Score}_i,$$

$$\text{IC Score}_i = \begin{cases} \frac{1}{2^{j-1}}, & \text{if } \mathbf{R}_i = \mathbf{S}_j, j \in \{1, 2, 3, 4\} \\ 0, & \text{otherwise} \end{cases}$$

The above formula indicates that weighted scores are given based on the number of times the model maintains Incremental Consistency. The higher the IC score, the higher Incremental Consistency of the model’s behavior. Specifically, for a model’s results of a problem:

- i) If the answers are completely correct, it gains a full score.
- ii) If the answers are partially correct and Incremental Consistency is observed, the model gains a partial score.
- iii) For other cases (e.g., partially correct, but Incremental Consistency is not observed), the model gets a zero score.

D. Benchmark Construction

As Fig. 2 illustrates, our framework utilizes existing executable benchmarks to evaluate the code reasoning ability of LLMs. We introduce how to adapt these base benchmarks into our framework in two steps: (i) Runtime Behavior Extraction; (ii) Problem Construction and Filtering:

1) *Runtime Behavior Extraction*: Our evaluation framework requires code models to predict intermediate information during code execution, thus, we need to extract the runtime information as the ground truth of the problem. We use the provided test case to execute the corresponding canonical solution to ensure the correctness of program and input. During the execution, we implement the customized program tracer to record (i) the statement being executed with its number of lines and (ii) the current program state (i.e., local variables) for each execution step. Thus, when the execution of code terminates, we can acquire an ordered sequence of the runtime behavior we need for the evaluation.

2) *Problem Construction and Filtering*: We construct our problem for each task with the extracted information. As there could be a large number of combinations of different runtime behavior and input (e.g., lots of variables in the program state of a specific time step), we design several filtering rules to select reasonable and representative ones:

C. 增量一致性评估

增量一致性（Incremental Consistency）指的是模型在一系列相互关联的任务中保持一致性的能力。直观而言，如果一个LLM无法对当前任务进行推理，则难以完成后续依赖于该任务的后续任务。为阐明这一概念，我们首先阐述增量一致性的定义，随后说明如何在实际代码模型中评估其表现。

1) 增量一致性的描述：核心思想

增量一致性的核心在于利用一系列任务中每个任务的知识都依赖于下一个任务这一关系来评估代码模型。在我们的研究背景下，我们选择了运行时行为推理中的四个不同子任务（即立方最密堆积、PSP、EPP和OP），并从中观察到了一些规律：

- i) **CPP PSP**：由于语句的执行可能导致程序状态发生变化，PSP的前提条件是能够准确预测该语句是否会被执行（即立方最密堆积）。
- ii) **PSP EPP**：运行程序的控制流程受某些变量值的影响（例如“if”分支及其条件变量），因此下一个要执行的语句（即EPP）取决于程序状态（即PSP）。
- iii) **EPP OP**：中间执行状态是影响程序输出的因素之一。因此，OP所需的知识涵盖了PSP所需的知识。

根据上述描述，我们发现完成前一项任务所需的知识实际上已包含在后续任务所需的知识中。从直觉上讲，后续任务的难度通常高于当前任务。因此，如果某个模型未能正确完成某项任务（例如未能完成CPP任务），但随后却能准确预测后续任务的结果（即输出预测正确），则我们认为该模型在连续执行任务时表现出不一致性。

2) 评估方法：我们分析了我们的研究结果。

运行时行为推理用于评估增量一致性。具体而言，对于我们的基准测试中的第*i*个特定问题（即完整程序、其中的特定语句、输入数据以及待提出的问题），我们假设四个任务的顺序结果如下：

$$\mathbf{R}_i = \{r_{\text{CPP}}, r_{\text{PSP}}, r_{\text{EPP}}, r_{\text{OP}}\},$$

其中 $r \in \{0,1\}$ ，该数值表示预测结果是否与真实值一致（即1）或不一致（即0）。因此，若模型具有增量一致性且仅在完成所有先前任务后才执行新任务，则二进制序列 $\mathbf{R}_i$ 应为非递减序列。

$\mathbf{R}_i \in \mathbf{S}$ ，其中 $\mathbf{S} = \{\{1,1,1,1\}, \{1,1,1,0\}, \{1,1,0,0\}, \{1,0,0,0\}\}$ 。

在图2所示的增量一致性评估首个示例中，生成的序列为 $\{1,1,0,0\}$ （E, E, e, e），表明在此场景中观察到了增量一致性。

然而，在第二个示例中，结果为 $\{0,1,1,0\}$ （e, E, E, e），因此未观察到增量一致性。

此外，根据一致性被维持的连续次数（例如 $\{1,1,1,1\}$ 表示为4次），我们会为更频繁保持增量一致性的模型分配不同的权重。这种设计具有直观性，因为在更多子任务中实现渐进式一致性更为困难。

最后，我们定义了**增量一致性得分**（IC Score），用于定量建模大语言模型*M*的增量一致性。对于包含*N*个 $(\mathbf{P}, \mathbf{X}, I, V)$ 数据对的基准测试集，IC得分可计算为：

$$\text{IC得分} = \frac{100}{N} \sum_{i=1}^N \text{IC得分}_i, \quad \text{IC得分}_i = \begin{cases} \frac{1}{2^{j-1}}, & \text{如果 } \mathbf{R}_i = \mathbf{S}_j, \text{ 则 } j \in \{1,2,3,4\}; \\ 0, & \text{否则} \end{cases}$$

上述公式表明，加权分数的计算基于模型保持增量一致性（IC）的次数。IC分数越高，说明模型行为的增量一致性越强。具体而言，针对模型在特定问题上的预测结果：

- i) 若答案完全正确，则可获得满分。
- ii) 若答案部分正确且符合增量一致性要求，则该模型可获得部分评分。
- iii) 对于其他情况（例如部分正确但未观察到增量一致性），该模型将获得零分。

D. 基准建设

如图2所示，我们的框架利用现有的可执行基准测试来评估大语言模型的代码推理能力。我们通过两个步骤将这些基础基准测试整合到框架中：(i)运行时行为提取；

(ii) 问题构建与过滤：

1) 运行时行为提取：我们的评估框架

代码执行过程需要依赖代码模型来预测执行过程中的中间信息，因此我们必须提取运行时信息作为问题的基准数据。我们使用提供的测试用例来执行相应的标准解决方案，以确保程序及输入数据的正确性。在执行过程中，我们实现了定制化的程序追踪器，用于记录：(i)正在执行的语句及其对应的行数；(ii)每个执行步骤下的当前程序状态（即局部变量值）。这样一来，在代码执行完成后，我们就能获得评估所需的一系列有序的运行时时行为序列。

2) **问题构建与过滤**：我们利用提取的信息为每个任务构建相应的问题模型。鉴于可能存在大量不同的运行时行为与输入组合（例如特定时间步程中程序状态包含众多变量），我们设计了多项过滤规则以筛选出合理且具有代表性的组合：

CCP and EPP. In these two tasks, we focus on whether and when a statement is being executed. As the actual execution sequence of statements could be very long for loops, we analyze the control flow graph and break the program into several blocks. We prioritize the last statement in a block as it leads to various new blocks and is generally more difficult to reason about.

PSP. This task challenges an LLM to predict the type and value of a variable. For **R**Eval, we inspect the code and focus on the following types of statements:

- i) **Assignment.** We extract the variable(s) at the left-hand side for assignment statements. The possible types of variables are identifier (i.e., ordinary variables like “x”), subscript (i.e., array slices like “x[0]”), and attribute (i.e., fields like “x.y”). In most cases, we extract identifiers. Note that some naive assignments such as `a = 0` or `l = []` are skipped, but we keep statements like `a += 1` for the change of variable value.
- ii) **Return statement.** In return statements, we extract the variables in the returned object if local variables are returned. If the returned object is a constant value, we will select the “nearest” variable, i.e., the last variable that is not constant.
- iii) **Others.** For other situations, if any variable after the current line is changed, we extract a changed variable based on the priority of “new variable > changed variables > changed attributes (i.e., `self.xxx`)”. Not all variables are used because we prefer variables that have closer logical relationship with other tasks (e.g., EPP). We ignore objects of non-serializable classes or complex structures (e.g., “self” objects), as it is challenging to convert them to canonical string representations and compare ground truth with an LLM’s output.

OP. In output prediction task, we follow CRUXEval [14] and utilize the assertion statements in the test cases of the base benchmarks. For base benchmarks such as ClassEval [22], where one test case contains multiple assertions, we use all the assertions. Specifically, we replace the right operands in the assertions with question marks (“?”), and challenge models to predict the masked values.

After the above screening, we combine the results to obtain the final adapted benchmark, ensuring that the dataset used for each task is consistent.

IV. EXPERIMENTAL SETUP

A. Base Benchmarks

In our experiments, we first need to obtain the runtime behavior of code such as program state, thus base benchmarks should be executable and equipped with test cases. Moreover, we would like to experiment with diverse types of data (e.g., different programming scenarios). Therefore, we utilize existing code generation benchmarks as the basis for code reasoning evaluation. Typically, code generation benchmarks can be categorized into two types: competition-level ones (i.e., with standalone functions) [9], [46] and context-aware ones

TABLE I
STATISTICS OF OUR DATASET.

Description	Number
# of Problems	3152
# of Avg. Tokens in Programs	408.3
# of Avg. Tokens in Selected Statements	14.0

TABLE II
FEATURES OF STUDIED LLMs. “FD”: FOUNDATION (CODE) MODELS. “IF”: SUPPORTING INSTRUCTION FOLLOWING. “OS”: OPEN-SOURCE MODELS.

Category	Series	Model Name	Size	FD	IF	OS	Time
Code LLMs	CodeLlama	CodeLlama-7B-Base	7B	✓	✗	✓	08/2023
		CodeLlama-7B-Python	7B	✓	✗	✓	08/2023
		CodeLlama-7B-Instruct	7B	✓	✓	✓	08/2023
		CodeLlama-13B-Instruct	13B	✓	✓	✓	08/2023
		CodeLlama-34B-Instruct	34B	✓	✓	✓	08/2023
	Magicoder	Magicoder-CL-7B	7B	✗	✓	✓	12/2023
		Magicoder-S-CL-7B	7B	✗	✓	✓	12/2023
	StarCoder2	StarCoder2-3B	3B	✓	✓	✓	02/2024
		StarCoder2-7B	7B	✓	✗	✓	02/2024
		StarCoder2-15B	15B	✓	✗	✓	02/2024
General LLMs	GPT	GPT-3.5-Turbo	-	-	✓	✗	01/2024
		GPT-4-Turbo	-	-	✓	✗	01/2024
	Mistral	Mistral-7B-Instruct	7B	✗	✓	✓	01/2024
	Gemma	Gemma-7B-It	7B	✓	✓	✓	02/2024
		Gemma-2B-It	2B	✓	✓	✓	02/2024

(i.e., with more code context like class and file dependencies) [22], [47]. Considering the diversity of base benchmarks, we choose **HumanEval** as a representative of competition-level benchmarks and **ClassEval** as a representative of context-aware benchmarks.

- **HumanEval** [8] is a popular competition-level benchmark for code generation. It consists of 164 hand-written Python programming problems and needs models to solve the problem given the function signature and docstring.
- **ClassEval** [22] is a class-level hand-written code generation benchmark. ClassEval provides different programming scenarios (e.g., incremental generation) and topics (e.g., management systems and database operations).

These benchmarks are widely used in previous research [15], [48], [49]. We show the statistics of the adapted benchmarks in Table I. In addition to our selection, we highlight that our framework is applicable for other similar code generation benchmarks.

B. Studied Code LLMs

To study the reasoning capabilities of diverse code LLMs, we curate a selection of models with a variety of distinctions. Specifically, we mainly consider these dimensions of them: (1) general or code specified (e.g., GPT-4-Turbo [50] v.s. CodeLlama [6]); (2) scale of parameters; (3) open-source or closed-source (e.g., GPT-3.5-Turbo [50] v.s. Mistral-7B-Instruct [51]); (4) foundation or further fine-tuned (e.g., CodeL-

立方最密堆积和EPP。在这两项任务中，我们重点研究语句是否以及何时被执行。由于循环结构中的实际执行序列可能非常长，我们分析控制流图并将程序划分为多个模块。我们优先考虑每个模块中的最后一个语句，因为该语句会引出多个新模块，且通常更难进行推理分析。

PSP.该任务要求大型语言模型预测变量的类型和值。对于**REval**，我们检查代码并重点关注以下类型的语句：

- i) **赋值操作。**我们从赋值语句的左侧提取变量。变量类型包括标识符（即普通变量如“x”）、下标（即数组切片如“x[0]”以及属性（即字段如“x.y”）。在大多数情况下，我们仅提取标识符。需要注意的是，诸如 a = 0 或 l = [] 这类简单的赋值操作会被跳过，但像 a += 1 这样用于改变变量值的操作则会被保留。
- ii) **返回语句。**在返回语句中，若返回的是局部变量，则会提取返回对象中的变量；若返回对象为常量值，则选择“最近”的变量（即最后一个非常量变量）。
- iii) **其他情况。**对于其他情形，若当前行之后的某个变量发生变化，我们将根据“新变量 > 已变更变量 > 已变更属性（即self.xxx）”的优先级来提取被变更的变量。并非所有变量都会被使用，因为我们更倾向于选择与其它任务（如EPP）具有更紧密逻辑关联性的变量。对于不可序列化类的对象或复杂结构（例如“self”对象），我们不予考虑，因为将其转换为标准字符串表示并与LLM输出结果进行比对存在较大困难。

引言。在输出预测任务中，我们遵循CRUXEval[14]的方法，并利用基础基准测试用例中的断言语句。对于ClassEval[22]等包含多个断言的基准测试用例，我们使用所有断言；具体而言，将断言中的右侧操作数替换为问号（“？”），并要求模型预测被遮蔽的数值。

经过上述筛选后，我们将各结果整合以获得最终适配的基准模型，并确保用于各项任务的数据集保持一致。

IV. 实验装置

A. 基本基准测试

在我们的实验中，首先需要获取代码的运行时行为（如程序状态），因此基准测试需具备可执行性并包含测试用例。此外，我们希望使用多种类型的数据进行实验（例如不同的编程场景）。为此，我们采用现有的代码生成基准测试作为代码推理评估的基础。通常，代码生成基准测试可分为两类：竞赛级基准测试（即包含独立函数的测试）[9]；[46] 以及具备上下文感知功能的系统。

表 I 我们数据集的统计信息。

描述	数量
问题数量	3152
程序中平均使用的令牌数量	408.3
选定语句中平均令牌的数量	14.0

表 II

所研究LLMS的特征：“FD”：基础（代码）模型；“IF”：后续支持性说明；“OS”：开源模型。

类别	系列	模型名称	尺寸	FD	如果；倘若	操作系统	时间
编码 法学 硕士	CodeLlama	CodeLlama-7B基础版					
		CodeLlama-7B（Python版本）					
		CodeLlama-7B（指令集版本）					
		CodeLlama-13B指令集					
	魔法编辑器	Magicoder-CL-7B	7B	✓	XX	✓	08/2023
		Magicoder-S-CL-7B	7B	✓	✓	✓	08/2023
			13B	✓	✓	✓	08/2023
			34B	✓	✓	✓	08/2023
			7B	XX	✓	✓	08/2023
			7B			✓	12/2023
							12/2023
	StarCoder2	StarCoder2-3B	3B	✓	✓X	✓	02/2024
		StarCoder2-7B	7B	✓	✓	✓	02/2024
		StarCoder2-15B	15B	✓	X	✓	02/2024
通用大型语言模型（LLM）	GPT	GPT-3.5-Turbo	-	-	✓	X	01/2024
		GPT-4-Turbo	-	-	✓	X	01/2024
	密斯特拉尔	Mistral-7B指令集	7B	X	✓	✓	01/2024
	杰玛	杰玛-7B-伊特	7B	✓	✓	✓	02/2024
		杰玛-2B-伊特	2B型	✓	✓	✓	02/2024

（即包含更多代码上下文信息，如类和文件依赖关系）

[22], [47]鉴于基准测试的多样性，我们选择HumanEval作为竞赛级基准的代表，而ClassEval则作为情境感知型基准的代表。

- **HumanEval**[8]是代码生成领域广受欢迎的竞赛级基准测试集，包含164道手工编写的Python编程问题，要求模型根据函数签名和文档字符串来解决问题。
- **ClassEval**[22]是一个面向类级别的手写代码生成基准测试工具。该工具提供了多种编程场景（如增量生成）和主题领域（如管理系统与数据库操作）。

这些基准指标在先前的研究中被广泛使用[15]。[48], [49]我们在表I中展示了适应性基准测试的统计结果。除我们的选择方案外，我们还需强调该框架同样适用于其他类似的代码生成基准测试。

B. 研究过的代码LLM模型

为研究不同代码生成型大语言模型（LLM）的推理能力，我们精选了一系列具有多样化特征的模型。具体而言，我们主要考察其以下维度：

- (1) 通用模型或特定代码（例如 GPT-4-Turbo [50] 与 CodeLlama [6]）；
- (2) 参数规模；
- (3) 开源或闭源（例如 GPT-3.5-Turbo [50] 与 Mistral-7BInstruct [51]）；
- (4) 基础模型还是经过进一步微调的模型（例如 CodeL-

You are given a Python function ...	<i>System Message</i>
[PYTHON] [/ANSWER]	<i>Few-shot Demonstrations</i>
[PYTHON] def f(x): f(3) [/PYTHON]	<i>Program</i>
[QUESTION] Is Line 3 (return x*2) executed when f(3) is called? [/QUESTION]	<i>Question</i>
[THOUGHT] 1. The function f is defined, which takes ... 2. ... [/THOUGHT]	<i>Thoughts (CoT only)</i>
[ANSWER] NO [/ANSWER]	<i>Answer</i>

Fig. 3. The prompt template for our empirical study. Note that the “Thoughts” part is used only we leverage Chain-of-Thought (CoT) [55] prompting.

lama v.s. Magicode-CL [7]); (5) instruct following or not (e.g., CodeLlama-7B-Base v.s. its instruct version); (6) open-source or not; and (7) release time.

As a result, considering these dimensions we select several state-of-the-art code LLMs that have been applied to various code related tasks [29], [52]–[54]. Table II presents detailed features of them and we can see that full consideration of the diversity of models across various features is taken to enhance the generalization of our study.

C. Prompt Design

In our work, we utilize prompting to evaluate code LLMs with our code reasoning tasks. We refer to a recent study [14] on code reasoning and design our prompt templates as illustrated in Fig. 3. For classic few-shot prompting, our prompt template consists mainly of five parts, including the system message, few-shot demonstrations, program, question, and answer. If Chain-of-Thought (CoT) [55] prompting is utilized, the thoughts are added into the prompt template as well as the examples in it. Both few-shot prompting and CoT prompting are widely applied in various tasks [55]–[57], including reasoning tasks [14].

D. Implementation Details

Access of Models and Base Benchmarks. For open-source models such as CodeLlama, we use the corresponding official releases available on HuggingFace [58]. For closed-source models (e.g., GPT-3.5-Turbo and GPT-4-Turbo), we invoke the OpenAI API [59] to access them. Two benchmarks (i.e., HumanEval and ClassEval) are also publicly available on HuggingFace [60], [61]. To help replicate our research, we list detailed information, such as model IDs and URLs in our replication package [62].

Environment. We run experiments on a Linux server with 8 NVIDIA A800 GPUs. For open-source LLMs, we deploy a local API server based on vLLM [63] which is a unified library for LLM serving and inference. All models are not quantized and we use their original precisions.

Configurations. Temperature can control the randomness in the generated results of models [8]. Specifically, we follow Gu et al. [14] and set the temperature to 0.8. For tasks with direct prompting, we set the maximum length of generated tokens to 256, while for tasks with CoT prompting, we set it to 1024. For the rest of the parameters, we use the default settings in vLLM, to ensure a fair comparison. To obtain reliable results, experiments for all open-source models with few-shot prompting are repeated five times, and we report the mean and standard deviation values in Section V. We do not repeat experiments for closed-source models like the GPT series due to a limited budget.

V. RESULTS

In this section, we discuss the results of our empirical study on $\mathcal{REval}$ by answering two research questions:

- **RQ1:** How do LLMs perform on Runtime Behavior Reasoning?
- **RQ2:** How do LLMs perform on Incremental Consistency evaluation?

A. RQ1: Performance of Runtime Behavior Reasoning

Table III shows the detailed results of Runtime Behavior Reasoning. All models are evaluated with few-shot prompting except for special annotated ones (i.e., CodeLlama-7b-Instruct (CoT)). Below, we discuss the results from different aspects.

Overall Performance. Overall, we find that the performance of different LLMs presents a large variation, and GPT-4-Turbo shows superior performance in reasoning about program execution. For example, GPT-4-Turbo achieves the best results in all metrics of Runtime Behavior Reasoning, and its average accuracy outperforms the second best (i.e., 55.7% of GPT-3.5) by a large margin (i.e., an absolute improvement of 19.3%). However, the overall performance of open-source models is not high and the best performer among them (i.e., CodeLlama-34B-Instruct) only achieves a level close to that of GPT-3.5 (i.e., 51.0% v.s. 55.7%) in terms of average accuracy.

Task. Runtime Behavior Reasoning consists of four distinct evaluation tasks, i.e., CCP, PSP, EPP, and OP, and the performance varies among different tasks. For example, all models achieve an accuracy of more than 50% in OP (i.e., Output Prediction), while only about half of them (i.e., 8 out of 15) can provide the correct answers for more than 10% problems in EPP (i.e., Execution Path Prediction). Hence, according to the average score of tasks, the performance distribution may suggest that EPP is the most challenging task and OP is relatively easy among them.

Size and Category. In general, we observe that for models within the same family, the variant with larger size of parameters shows better performance in Runtime Behavior Reasoning. In the case of the CodeLlama-instruct series, as the number of parameters increases (i.e., 7B $\rightarrow$ 34B), the accuracy of EPP has a relative improve by over 100% (i.e., 14.4% $\rightarrow$ 29.2%). Meanwhile, smaller models like StarCoder2-3B

您被提供了一个Python函数.....		系统消息
[Python] [无答案]		少量案例演示
[Python] def f(x): f(3) [/根目录]		程序
[问题]: 第3号线是吗? 当调用 f(3) 时, 会执行 return x*2) 吗? [结束提问]		问题: 疑问
[思考] 1. 函数 f 已定义, 其参数为..... 2. ... [思考]		想法 (仅限 CoT)
[答案] 不 [无答案]		答案

图3. 我们实证研究的提示模板。请注意，“思考”部分仅用于我们采用思维链（CoT）[55]提示法时。

(1) 与Magicoder-CL相比[7]; (2) 是否包含指令功能（例如CodeLlama-7B-Base与其指令版本对比）; (3) 是否开源; (4) 发布时间。

因此，综合考虑这些维度后，我们选择了若干已应用于各类代码相关任务的先进代码大语言模型[29]。[52]–[54]。表II详细列出了这些模型的特征，可见我们充分考虑了不同特征下模型的多样性，从而提升了本研究的泛化能力。

C. 即时设计

在我们的工作中，我们利用提示机制来通过代码推理任务评估代码大语言模型。我们参考了近期关于代码推理的研究[14]，并设计出如图3所示的提示模板。对于经典的少样本提示方法，我们的提示模板主要包含五个部分：系统提示语、少量示例演示、程序代码、问题及答案。若采用思维链（CoT）[55]提示方法，则会在提示模板中加入思考过程及其示例。少样本提示与思维链提示均已被广泛应用于各类任务[55]–[57]，包括推理任务[14]。

D. 实施细节

模型与基准测试集的获取。对于CodeLlama等开源模型，我们使用HuggingFace平台提供的官方版本[58]；而对于闭源模型（如GPT-3.5-Turbo和GPT-4-Turbo），则通过OpenAI API[59]进行调用。此外，两项基准测试集（即HumanEval和ClassEval）亦在HuggingFace平台上公开提供[60]。[61]为便于他人复现我们的研究，我们在复现包[62]中列出了详细信息（如模型ID和URL）。

环境。我们在配备8块 NVIDIA A800 GPU的Linux服务器上进行实验。针对开源大语言模型（LLMs），我们部署了基于vLLM[63]构建的本地API服务器——该库是专为LLM服务与推理设计的统一平台。所有模型均未量化处理，我们直接采用其原始精度参数。

配置。温度能够控制模型生成结果中的随机性[8]。具体而言，我们遵循Gu等人的方法。[14] 并将温度设置为0.8。对于需要直接提示的任务，我们将生成的令牌最大长度设为对于使用CoT提示的任务，我们将参数值设为1024；其余参数均采用vLLM的默认设置以确保公平比较。为获得可靠结果，所有采用少样本提示的开源模型实验均重复五次，并在V节中报告平均值与标准差；由于预算限制，我们未对GPT系列等闭源模型进行重复实验。

V. 结果

在本节中，我们通过回答两个研究问题，讨论了关于REval的实证研究结果：

- **RQ1:** 大语言模型在运行时行为推理方面的表现如何？
- **RQ2:** 大语言模型在增量一致性评估中的表现如何？

A. RQ1: 运行时行为推理的性能表现

表III展示了运行时行为推理的详细结果。除特殊标注模型（即CodeLlama-7b-Instruct（CoT））外，所有模型均采用少样本提示进行评估。下文将从不同角度讨论这些结果。

总体性能。综观各项结果，我们发现不同大语言模型的性能存在显著差异，其中GPT-4-Turbo在程序执行推理方面表现尤为突出。例如，在运行时行为推理的所有评估指标中，GPT-4-Turbo均取得最佳成绩；其平均准确率较第二名（即GPT-3.5的55.7%）大幅领先（绝对提升幅度达19.3%）。然而开源模型的整体性能仍较低，其中表现最佳者（CodeLlama-34B-Instruct）在平均准确率方面仅接近GPT-3.5水平（51.0% vs 55.7%）。

任务。运行时行为推理包含四个不同的评估任务，即立方最密堆积、PSP、EPP和OP，其性能在不同任务间存在差异。例如，所有模型在OP（即输出预测）任务中均能达到超过50%的准确率；而在EPP（即执行路径预测）任务中，仅约半数模型（15个中的8个）能对超过10%的问题给出正确答案。因此，根据各任务的平均得分来看，性能分布表明EPP是其中最具挑战性的任务，而OP相对较为简单。

规模与类别。总体而言，我们观察到在同一模型家族中，参数规模较大的变体在运行时行为推理方面表现更优。以CodeLlama-instruct系列为例，随着参数数量增加（即从70亿→340亿），EPP的准确率相对提升了超过100%（即从14.4%→29.2%）。与此同时，较小规模的模型如StarCoder2-3B

TABLE III
RESULTS FOR RUNTIME BEHAVIOR REASONING AND INCREMENTAL CONSISTENCY EVALUATION (RQ1 & 2). “CCP”, “PSP”, “EPP”, AND “OP”: FOUR TASKS OF RUNTIME BEHAVIOR REASONING; “AVG”: THE AVERAGE ACCURACY SCORE OF FOUR TASKS. WE REPORT THE RESULTS IN THE FORM OF “MEAN±STANDARD DEVIATION” EXCEPT FOR TWO GPT MODELS BECAUSE OF BUDGET LIMIT.

Model	CCP		PSP	EPP	OP	Acc. Avg.	IC
	Acc. (%)	F1	Acc. (%)	Acc. (%)	Acc. (%)	(%)	Score
CodeLlama-7B-Base	54.3±0.5	56.1±0.5	25.0±0.5	5.6±0.3	58.2±1.6	35.8	4.0±0.2
CodeLlama-7B-Python	55.5±0.7	62.7±0.6	31.3±1.0	8.7±0.7	62.3±1.0	39.4	4.8±0.2
CodeLlama-7B-Instruct	55.6±0.9	47.2±1.2	25.1±0.5	10.8±0.2	62.6±0.8	38.5	4.1±0.1
CodeLlama-13B-Instruct	61.0±0.7	66.4±0.6	32.5±0.4	14.4±0.4	64.5±1.1	43.1	6.6±0.3
CodeLlama-34B-Instruct	61.5±0.5	70.1±0.4	47.5±0.6	29.2±0.4	65.9±1.1	51.0	11.8±0.3
StarCoder2-3B	54.8±0.7	58.2±0.5	29.0±0.7	6.5±0.5	58.8±0.8	37.3	4.3±0.3
StarCoder2-7B	55.1±0.7	63.8±0.6	34.2±0.7	5.0±0.4	63.9±0.8	39.6	4.2±0.3
StarCoder2-15B	58.9±0.8	64.6±0.8	43.5±0.3	28.0±0.5	71.5±1.1	50.5	10.7±0.4
MagiCoder-CL	58.7±1.4	61.2±1.8	30.1±0.5	15.5±1.1	60.4±1.4	41.2	6.2±0.3
MagiCoder-S-CL	60.3±1.1	69.9±0.8	31.4±0.4	9.8±0.4	62.3±1.2	40.9	6.0±0.2
Gemma-2B-It	52.7±0.4	31.0±0.6	13.5±0.5	7.3±0.5	43.9±1.5	29.3	5.5±0.2
Gemma-7B-It	66.3±0.3	75.2±0.1	32.1±0.1	8.4±0.4	57.9±0.7	41.2	6.9±0.2
Mistral-7B-Instruct	69.5±0.2	75.9±0.2	35.2±0.3	35.8±0.4	51.5±0.7	48.0	16.3±0.3
GPT-3.5-Turbo	61.8	64.0	51.6	48.6	60.7	55.7	20.6
GPT-4-Turbo	88.4	89.8	71.4	57.7	82.6	75.0	42.5
Average	61.0	63.7	35.6	19.4	61.8	44.4	10.3
CodeLlama-7B-Instruct (CoT)	57.5	59.2	33.4	21.4	55.8	42.2	7.5

can also outperform larger models such as CodeLlama-7B-Instruct in terms of average accuracy (i.e., 37.3% v.s. 35.8%). The StarCoder2 series utilizes varied architectures and training datasets compared to CodeLlama. This may demonstrate that apart from parameter size, the model architecture and training strategy also play an important role in code reasoning ability. We also find that code LLMs do not exhibit an obviously leading advantage over general LLMs of the same size.

Training Strategy. As shown in Table III, we conduct experiments on three variants of CodeLlama (i.e., base, instruct, and Python) of the same 7B size. Compared to the base model, the “instruct” variant that leverages instruction tuning techniques brings gains in the code reasoning ability (i.e., Avg. Acc. from 35.8% to 38.5%), which may reflect the relationship between understanding instructions and reasoning program. Meanwhile, since our base benchmarks are all in Python, additional training with Python corpora (i.e., CodeLlama-Python) leads to an improvement of performance (i.e., an absolute improvement of 0.9%). In addition, we note that although further fine-tuning applied to the MagiCoder series improves their performance in code generation [7], the improvement in code reasoning ability is relatively limited compared to their foundation model CodeLlama-7B-Python. This may indicate that the training strategies they utilize are not well suited for the reasoning tasks in our evaluation.

Prompting Strategy. The last row of Table III presents the performance of CodeLlama-7B-Instruct with CoT prompting. Compared to the model with few-shot prompting, the perfor-

mance of CPP, PSP, and EPP receives varying degrees of improvement. For instance, the EPP accuracy with CoT is improved from 10.8% to 21.4%, surpassing the performance of the larger 13B model (i.e., 14.4%). It may demonstrate the effectiveness of presenting how to reason about a piece of code step by step. However, CoT prompting fails to improve its OP performance, with an absolute decrease of 6.8%. This may result from the wrong thought chain that the model generates for the whole program and eventually leads to the mistake.

Summary for RQ1: Models with different features (e.g., size and training strategy) exhibit notable disparities in performance on Runtime Behavior Reasoning. Overall, GPT-4-Turbo demonstrates a clear advantage over other models in all four tasks in our setting.

B. RQ2: Incremental Consistency Evaluation

Fig. 4 shows the sorted average accuracy for Runtime Behavior Reason for different models, with an additional line indicating the IC scores. A detailed information of mean IC scores and their standard deviations are also reported in Table III.

Overall Performance. We find that the majority of LLMs exhibit a low level of Incremental Consistency with scores below 20, which highlights the inconsistency in model behavior across the four tasks of Runtime Behavior Reasoning. Among all of the models, GPT-4-Turbo stands out with the highest IC score of 42.5, even more than double that of the second place

表 III
运行时行为推理与增量一致性评估（RQ1 & 2）的结果。“立方最密堆积”、“PSP”、“EPP”及“OP”：四项运行时行为推理任务；“AVG”：四项任务的平均准确率得分。
除两种GPT模型因预算限制外，其余结果均以“均值±标准差”形式呈现。

模型	中国共产党		掌上游戏机	欧洲人民党	操作；处理	平均账户余额。	IC
	账户余额（%）	F1 账户余额（%）	账户余额（%）	账户余额（%）	（%）	（%）	分数；得分
CodeLlama-7B-Base	54.3±0.5	56.1±0.5	25.0±0.5	5.6±0.3	58.2±1.6	35.8	4.0±0.2
CodeLlama-7B-Python	55.5±0.7	62.7±0.6	31.3±1.0	8.7±0.7	62.3±1.0	39.4	4.8±0.2
CodeLlama-7B指令集	55.6±0.9	47.2±1.2	25.1±0.5	10.8±0.2	62.6±0.8	38.5	4.1±0.1
CodeLlama-13B指令集	61.0±0.7	66.4±0.6	32.5±0.4	14.4±0.4	64.5±1.1	43.1	6.6±0.3
CodeLlama-34B指令集	61.5±0.5	70.1±0.4	47.5±0.6	29.2±0.4	65.9±1.1	51.0	11.8±0.3
StarCoder2-3B	54.8±0.7	58.2±0.5	29.0±0.7	6.5±0.5	58.8±0.8	37.3	4.3±0.3
StarCoder2-7B	55.1±0.7	63.8±0.6	34.2±0.7	5.0±0.4	63.9±0.8	39.6	4.2±0.3
StarCoder2-15B	58.9±0.8	64.6±0.8	43.5±0.3	28.0±0.5	71.5±1.1	50.5	10.7±0.4
魔导者-CL	58.7±1.4	61.2±1.8	30.1±0.5	15.5±1.1	60.4±1.4	41.2	6.2±0.3
Magicode-S-CL	60.3±1.1	69.9±0.8	31.4±0.4	9.8±0.4	62.3±1.2	40.9	6.0±0.2
杰玛-2B-伊特	52.7±0.4	31.0±0.6	13.5±0.5	7.3±0.5	43.9±1.5	29.3	5.5±0.2
杰玛-7B-伊特	66.3±0.3	75.2±0.1	32.1±0.1	8.4±0.4	57.9±0.7	41.2	6.9±0.2
米斯特拉尔-7B型导弹的使用说明	69.5±0.2	75.9±0.2	35.2±0.3	35.8±0.4	51.5±0.7	48.0	16.3±0.3
GPT-3.5-Turbo	61.8	64.0	51.6	48.6	60.7	55.7	20.6
GPT-4-Turbo	88.4	89.8	71.4	57.7	82.6	75.0	42.5
平均值	61.0	63.7	35.6	19.4	61.8	44.4	10.3
CodeLlama-7B-Instruct（代码训练）	57.5	59.2	33.4	21.4	55.8	42.2	7.5

在平均准确率方面，该模型甚至能超越CodeLlama-7B-Instruct等大型模型（分别为37.3%和35.8%）。与CodeLlama相比，StarCoder2系列采用了多种不同的架构和训练数据集，这表明除参数规模外，模型架构和训练策略对代码推理能力也起着关键作用。此外我们发现，代码生成型大语言模型并未在同等规模的通用大语言模型中展现出明显优势。

训练策略。如表III所示，我们在相同7B规模的CodeLlama三个变体（即基础版、指令版和Python版）上进行了实验。与基础模型相比，采用指令调优技术的“指令版”在代码推理能力方面取得了提升（平均准确率从35.8%提高至38.5%），这可能反映了理解指令与程序推理之间的关联。同时，由于我们的基准测试均基于Python数据集，使用Python语料库进行额外训练（即CodeLlama-Python）可进一步提升性能（绝对提升幅度达0.9%）。此外我们注意到，尽管对Magicoder系列模型进行进一步微调能提升其代码生成性能[7]，但其代码推理能力的提升相较于基础模型CodeLlama-7B-Python而言相对有限，这表明其所采用的训练策略并不完全适用于本评估中的推理任务。

提示策略。表III的最后一行展示了CodeLlama-7B-Instruct在CoT提示下的性能表现。与采用少样本提示的模型相比，其性能表现.....

CPP、PSP和EPP模型的各项性能均有所提升。例如，采用代码解释（CoT）技术后，EPP模型的准确率从10.8%提升至21.4%，超越了规模更大的13B模型（准确率为14.4%）。这表明逐步展示代码推理过程的方法具有显著效果。然而，代码解释提示并未改善其最终预测性能，反而导致准确率下降6.8%。这可能是由于模型为整个程序生成了错误的推理链路，从而导致错误结果。

RQ1总结：具有不同特征（如规模和训练策略）的模型在运行时行为推理任务上的性能存在显著差异。总体而言，在我们的实验设置中，GPT-4-Turbo在所有四项任务中均明显优于其他模型。

B. RQ2：增量一致性评估

图4展示了不同模型在运行时行为原因方面的排序平均准确率，并额外添加了一条线表示IC分数。平均IC分数及其标准差的详细信息亦列于表III中。

总体性能。我们发现大多数大语言模型（LLM）的增量一致性（IC）得分均低于20，表明其在运行时行为推理的四项任务中表现存在显著不一致。在所有模型中，GPT-4-Turbo表现最为突出，其IC得分为42.5，是第二名模型得分的两倍以上。

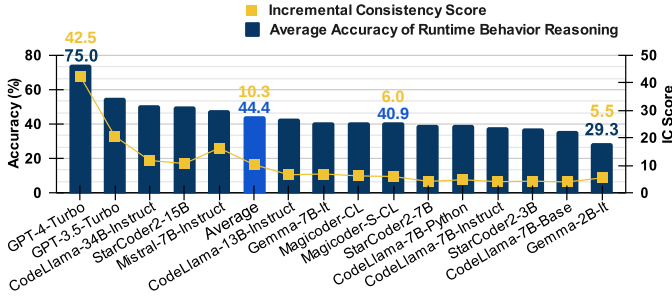


Fig. 4. Average Accuracy of Runtime Behavior Reasoning and Incremental Consistency Score for different models, sorted in descending order according to average accuracy.

GPT-3.5 (i.e., 20.6). Given that GPT-4-Turbo achieves the best results in both Runtime Behavior Reasoning and Incremental Consistency Evaluation (i.e., 75.0% of Avg. Acc. and 42.5 of IC score), we believe that it has both superior ability in program reasoning and a high level of Incremental Consistency across sequentially related tasks in our evaluation.

Trend Between IC and Runtime Behavior Reasoning. As illustrated in Fig. 4, we find that there is an approximately similar trend between the model’s average accuracy and its IC score. For example, compared to CodeLlama-7B-Instruct, its larger version CodeLlama-34B-Instruct has a noticeable higher average accuracy (i.e., 38.5% v.s. 51.0%) and IC score (i.e., 4.1 v.s. 11.8). However, this pattern does not hold for all models. The performance of the general LLM Mistral-7B is not as good as that of CodeLlama-34B in terms of average accuracy (i.e., 48.0% v.s. 51.0% in Runtime Behavior Reasoning), but performs better on Incremental Consistency (i.e., 16.3 v.s. 11.8).

Others. Similar to the results of RQ1, we observe that **code LLMs do not significantly outperform general LLMs in the IC evaluation**. For example, the IC score of Gemma-2B-It (i.e., 5.5) is higher than code LLMs trained with more code corpora like CodeLlama-7B-Instruct and StarCoder2-7B (i.e., 4.1 and 4.2). This phenomenon may suggest that more code data cannot help LLMs reason programs better and maintain their Incremental Consistency. In addition, CoT prompting for CodeLlama-7B-Instruct leads to a great increase in its IC Score (i.e., 4.1 $\rightarrow$ 7.5), and this improvement of IC Score may benefit from explicit problem solving steps.

Summary for RQ2: In code reasoning tasks, **most LLMs behave inconsistently and their average accuracy is not entirely associated with IC**. GPT-4-Turbo achieves an IC Score of as high as 42.5, surpassing other models by a large margin (i.e., more than 21.9 absolute improvements).

VI. DISCUSSION

A. Case Study

Fig. 5 shows a case from the problem of EPP. Given a Python function that aims to return the largest prime factor of

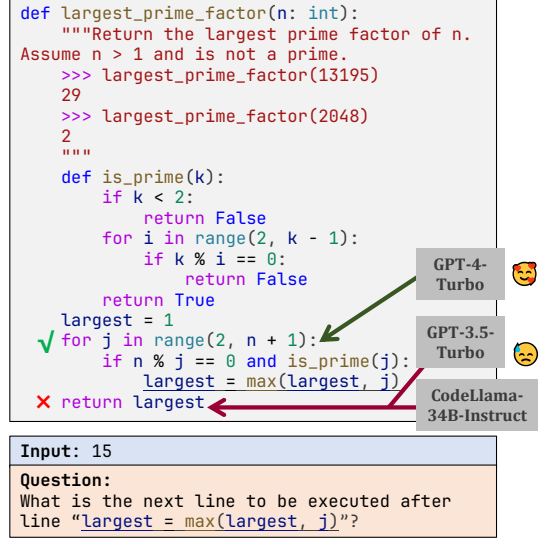


Fig. 5. A tricky problem of EPP from HumanEval/59 [8]. The prediction of GPT-4-Turbo is correct and the other two models (i.e., GPT-3.5-Turbo and CodeLlama-34B-Instruct) fail to finish it. The problem description is simplified for a concise presentation.

parameter n and input 15, this problem requires the model to predict the next statement to be executed after an assignment statement (i.e., `largest = max ...`). Here we select three models that show competitive performance in EPP including GPT-4-Turbo, GPT-3.5-Turbo, and CodeLlama-34B-Instruct. For GPT-3.5-Turbo and CodeLlama-34B-Instruct, the prediction is the last “return” statement of this function; while GPT-4-Turbo chooses the above “for” loop as its prediction. We mark a happy emoji to show that GPT-4-Turbo makes the right choice, and the other two models fail to predict it correctly. The explanation is that if the assignment statement is executed, the possible value of j can only be 3 or 5 ($n+1 = 16$), which means that the loop will continue and the “return” statement is not the next executed line.

In our framework and the adapted benchmark, there are many problems like this that have no obvious answer and are challenging. If the model is not capable of reasoning about its inherent logic of execution, it can easily be misled and give the most “look-alike” answer (i.e., the “return” statement in this case), indicating that our framework can effectively measure the code reasoning capability of LLMs and present the discrimination of them. We discuss more cases in the appendix, which can be accessed in our replication package¹.

B. Unsatisfactory Performance of Code Reasoning

According to evaluation results, we observe that many models perform poorly in Runtime Behavior Reasoning and Incremental Consistency Evaluation. In particular, even the best performer GPT-4-Turbo only achieves an IC score of 42.5, reflecting the limitation of current models in maintaining consistency in sequential-related tasks, and there is still a long way to go to make the LLMs perform code reasoning.

¹<https://r-eval.github.io>

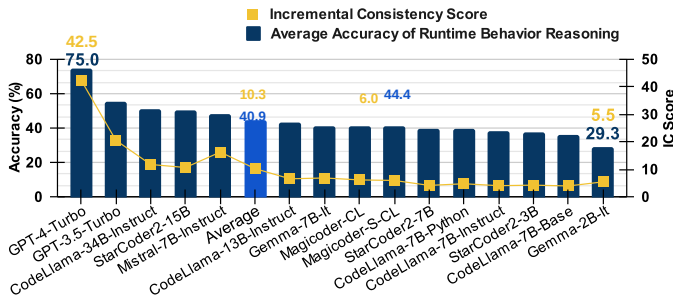


图4. 不同模型的运行时行为推理与增量一致性评分的平均准确率（按平均准确率降序排列）。

GPT-3.5（即版本20.6）。鉴于GPT-4-Turbo在运行时行为推理和增量一致性评估两项指标上均取得最佳成绩（平均准确率达75.0%，增量一致性得分达42.5分），我们认为其不仅具备卓越的程序推理能力，在本次评估中对序列相关任务也展现出高水平的增量一致性。

IC值与运行时行为推理能力之间的趋势。如图4所示，我们发现模型的平均准确率与其IC值之间呈现出大致相似的趋势。例如，与CodeLlama-7B-Instruct相比，其更大规模版本CodeLlama-34B-Instruct的平均准确率（38.5% 对比 51.0%）和IC值（4.1 对比 11.8）均显著更高。然而，这一规律并非适用于所有模型：通用大语言模型Mistral-7B在运行时行为推理方面的平均准确率（48.0% 对比 51.0%）虽不及CodeLlama-34B，但在增量一致性测试中表现更优（16.3 对比 11.8）。

其他发现。与RQ1的结果类似，我们观察到代码生成型大语言模型在增量一致性（IC）评估中并未显著优于通用大语言模型。例如，Gemma-2B-It的IC得分（5.5）高于使用更多代码语料库训练的代码生成型大语言模型（如CodeLlama-7b-Instruct和StarCoder2-7B，得分分别为4.1和4.2）。这一现象可能表明，更多的代码数据并不能帮助大语言模型更有效地推理程序或保持其增量一致性。此外，对CodeLlama-7B-Instruct进行代码理论（CoT）提示后，其IC得分显著提升（从4.1 → 7.5），这种提升可能得益于明确的问题解决步骤。

RQ2 总结：在代码推理任务中，大多数大语言模型表现不稳定，其平均准确率与信息一致性（IC）并非完全相关。GPT-4-Turbo的IC得分高达42.5，远超其他模型（绝对提升幅度超过21.9）。

VI. 讨论

A. 案例研究

图5展示了EPP问题中的一个实例。给定一个Python函数，其目标是返回最大的质因数。

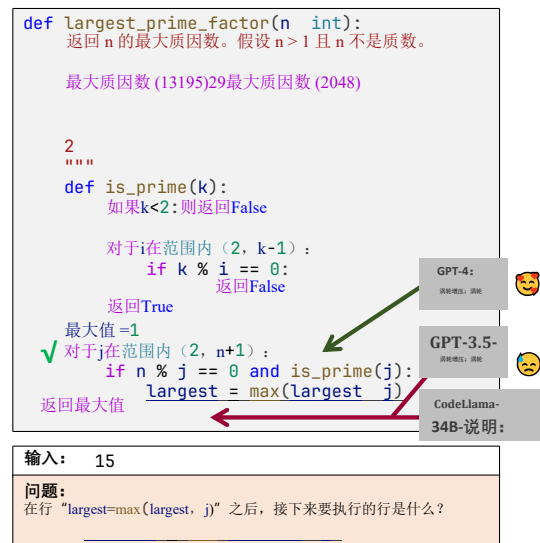


图5. 来自HumanEval/59的EPP难题[8]。GPT-4-Turbo的预测结果正确，而另外两个模型（即GPT-3.5-Turbo和CodeLlama-34B-Instruct）均未能完成该任务。为简洁呈现，问题描述已简化。

对于参数 n 和输入 15，该问题要求模型预测赋值语句（即 `largest = max`）之后应执行的下一条语句。我们选取了在 EPP 测试中表现优异的三个模型：GPT-4-Turbo、GPT-3.5-Turbo 和 CodeLlama-34B-Instruct。对于 GPT-3.5-Turbo 和 CodeLlama-34B-Instruct，其预测结果是该函数的最后一行“return”语句；而 GPT-4-Turbo 则选择了上述“for”循环作为预测结果。我们使用笑脸表情符号表示 GPT-4-Turbo 的预测正确，另外两个模型均未能准确预测。原因在于：若执行赋值语句，则 j 的可能取值仅为 3 或 5 ($n+1 = 16$)，这意味着循环将继续执行，“return”语句并非下一个被执行的代码行。

在我们的框架及相应的基准测试中，存在许多此类问题——这些问题没有明确的答案且极具挑战性。如果模型无法对其执行过程中的内在逻辑进行推理，就很容易被误导而给出最“相似”的答案（本例中即为“return”语句），这表明我们的框架能够有效评估大语言模型的代码推理能力并揭示其差异性。我们在附录中讨论了更多案例，相关内容可参见我们的复现包¹。

B. 代码推理性能不佳

评估结果显示，许多模型在运行时行为推理和增量一致性评估方面表现不佳。尤其是性能最佳的GPT-4-Turbo，其增量一致性（IC）得分仅为42.5，这反映出当前模型在处理序列相关任务时难以保持一致性；要让大型语言模型（LLM）具备代码推理能力，仍有很长的路要走。

¹<https://r-eval.github.io>

TABLE IV
PEARSON CORRELATION COEFFICIENT MATRIX OF THE RESULTS OF
RUNTIME BEHAVIOR REASONING (RBR), INCREMENTAL CONSISTENCY
(IC), AND HUMANEval (HE)

Pearson Correlation	RBR	IC	HE
RBR	1.000	0.940	0.772
IC	0.940	1.000	0.724
HE	0.772	0.724	1.000

One potential reason is that the current LLM might not understand the program execution behavior. While it is convenient to obtain source code from open-source platforms (e.g., GitHub), there is relatively less data available regarding code execution behavior, because running the program and collecting its runtime information require the corresponding development environments and test suite. Therefore, if the model is not familiar with the knowledge related to runtime behavior, it may not perform well in code reasoning tasks.

C. Correlation between Code Reasoning & Code Generation

We utilize the experimental results and study the correlation between code reasoning and code generation, i.e., whether an LLM that performs well in code generation could exhibit equally strong abilities in code reasoning.

Table IV presents the Pearson correlation coefficient matrix of Runtime Behavior Reasoning (Avg. Acc.), Incremental Consistency Score and HumanEval (*pass@1* rate). According to the matrix, we find that there is a strong positive correlation (i.e., the Pearson correlation coefficients are larger than 0.7) among code generation (i.e., HumanEval) and code reasoning (i.e., Runtime Behavior Reasoning and Incremental Consistency). However, the correlation between code reasoning and code generation is relatively lower than that between two reasoning tasks internally (i.e., 0.724 and 0.772 v.s. 0.940), which indicates that models with similar code generation abilities may vary a lot in code reasoning. As the correlation may help researchers increase the understanding of code LLMs and improve the models' code generation and reasoning abilities, future research could investigate such correlation in depth.

D. Threats to Validity

Internal Threats. To construct the adapted benchmark for different sub-tasks of Runtime Behavior Reasoning, we manually establish some rules to select appropriate statements and variables for the evaluation. However, our selection criteria may not effectively represent the runtime state of the program. To mitigate this threat, we take some measures to determine problem settings that are representative and challenging, based on the characteristics of different tasks. For instance, we choose the last statement in the control flow (i.e., for EPP) and variables that are modified after the execution (i.e., for PSP). These measures help us to reasonably assess the model's capability to reason about code and provide meaningful differentiation. For Runtime Behavior Reasoning, we select four dimensions of the intermediate state of program execution

which are widely applied in previous research [16], [42]. These four tasks are proven to effectively evaluate the code reasoning capability of code LLMs [15], [16], [42], and are appropriate for Incremental Consistency Evaluation for their unique sequential relationship. However, there are still some dynamic features such as memory allocation and exception handling which may help measure code models, and we have not explored yet. Further research could consider exploring the potential for LLMs to reason about other dynamic program features and extend $\mathcal{REval}$ to more scenarios.

External Threats. In the empirical study for our evaluation framework, the results are restricted to the specific collection of code models and base benchmarks. To mitigate this threat, we choose representative code LLMs considering several standards including their scale, popularity, and training strategy; For the base benchmarks applied, two benchmarks (i.e., HumanEval [8] and ClassEval [22]) are distinct from evaluation fashion and programming scenarios, as described in Section III-D. With the above efforts, the experimental results are expected to be sustained in more circumstances.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose $\mathcal{REval}$, a comprehensive framework for evaluating the code reasoning capability of code LLMs. Our framework consists of two evaluation components including Runtime Behavior Reasoning and Incremental Consistency Evaluation: We conduct a large-scale empirical study on several popular LLMs and two widely used base benchmarks. Our empirical results show that the majority of LLMs we evaluate show unsatisfactory performance in both Runtime Behavior Reasoning and Incremental Consistency Evaluation. To improve the code reasoning capabilities of LLMs, future works can explore:

Training with Execution Behavior. One reason why large models may struggle with code reasoning is possibly due to a lack of knowledge related to program execution. Although some general fine-tuning approaches are applied to code LLMs [7], they fail to improve the code reasoning capabilities. Given the demonstrated effectiveness of training models with execution behavior in improving performance in a range of downstream tasks [16], [24], it is reasonable to expect that LLMs would also derive benefits from such a process.

Improving Prompting Strategy. Our evaluation results demonstrate the effectiveness of CoT prompting in code reasoning tasks. Apart from CoT, other prompting techniques that have been proven effective in NL reasoning tasks (such as Tree-of-Thoughts [64]) may also be applicable to code reasoning tasks. Besides, the prompting approach tailored for reasoning about program execution also warrants investigation.

ACKNOWLEDGMENTS

This research is supported by the Ningbo Natural Science Foundation (No. 2023J292). It is also supported by the advanced computing resources provided by the Supercomputing Center of Hangzhou City University.

表 IV
运行时行为推理（RBR）与增量一致性结果的皮尔逊相关系数矩阵（IC），以及humaneval（HE）

皮尔逊相关系数	橡胶支座；橡胶衬垫	IC	HE
橡胶支座；橡胶衬垫	1.000	0.940	0.772
IC	0.940	1.000	0.724
HE	0.772	0.724	1.000

一个潜在原因在于，现有的大语言模型可能无法理解程序的执行行为。虽然从开源平台（如GitHub）获取源代码十分便捷，但关于代码执行行为的数据相对匮乏——因为运行程序并收集其运行时信息需要相应的开发环境和测试套件。因此，如果模型缺乏对运行时行为相关知识的理解，其在代码推理任务中的表现可能会大打折扣。

C. 代码推理与代码生成之间的相关性

我们利用实验结果研究了代码推理与代码生成之间的相关性，即在代码生成方面表现优异的大型语言模型（LLM）是否也能在代码推理方面展现出同等强大的能力。

表IV展示了运行时行为推理（平均准确率）、增量一致性得分与HumanEval（通过率）之间的皮尔逊相关系数矩阵。根据该矩阵可知，代码生成（即HumanEval）与代码推理（即运行时行为推理和增量一致性）之间存在强正相关关系（即皮尔逊相关系数均大于0.7）。然而，代码推理与代码生成之间的相关性相对低于两个推理任务内部的相关性（分别为0.724、0.772对比0.940），这表明具有相似代码生成能力的模型在代码推理方面可能存在显著差异。鉴于相关性分析有助于研究人员深入理解代码大语言模型并提升其代码生成与推理能力，未来研究可对此类相关性进行更深入探讨。

D. 对研究有效性的威胁

内部威胁。为构建适用于运行时行为推理各项子任务的适应性基准，我们手动制定了若干规则以筛选适合评估的语句和变量。然而，我们的选择标准可能无法充分反映程序的实际运行状态。为缓解这一风险，我们根据不同任务的特点，采取了相应措施来确定具有代表性且具有挑战性的测试场景：例如，在控制流中选取最后一条语句（适用于EPP）以及执行后被修改的变量（适用于PSP）。这些措施有助于我们合理评估模型对代码进行推理的能力，并实现有效的性能区分。针对运行时行为推理任务，我们选取了程序执行过程中中间状态的四个维度作为评估指标。

这些方法已在先前的研究中得到广泛应用[16]。[42]这四项任务已被证实能有效评估代码大语言模型（LLM）的代码推理能力[15]。[16]，[42]这些方法因其独特的序列关系，非常适合用于增量一致性评估。然而，其中仍存在内存分配和异常处理等动态特性，这些特性有助于衡量代码模型，但我们尚未深入探讨。后续研究可进一步探索大型语言模型（LLMs）对其他动态程序特征进行推理的能力，并将REval的应用场景扩展到更多领域。

外部威胁。在我们评估框架的实证研究中，结果仅限于特定的代码模型集合和基础基准测试集。为缓解这一风险，我们选取了具有代表性的代码大语言模型（LLM），综合考虑其规模、普及度及训练策略等标准；所采用的基础基准测试中，HumanEval[8]与ClassEval[22]两项指标分别针对不同的评估方式和编程场景设计，具体详见III-D节。通过上述措施，实验结果有望在更多应用场景中保持稳定可靠性。

VII. 结论与未来研究方向

本文中，我们提出了REval——一个用于评估代码大语言模型代码推理能力的综合性框架。该框架包含两个评估模块：运行时行为推理与增量一致性评估；我们针对多个主流大语言模型及两项广泛使用的基准测试进行了大规模实证研究。实验结果表明，在运行时行为推理和增量一致性评估两个维度上，绝大多数被评估的大语言模型均表现不佳。为提升大语言模型的代码推理能力，未来研究可重点探索以下方向：

基于执行行为的训练。大型模型在代码推理方面表现不佳的一个原因，可能在于其缺乏与程序执行相关的知识。尽管已有一些通用微调方法被应用于代码生成式大语言模型[7]，但这些方法仍无法提升其代码推理能力。鉴于基于执行行为的模型训练已被证实能显著提升多种下游任务的性能[16]，[24]有理由认为，大型语言模型（LLM）也将从这一过程中获益。

优化提示策略。我们的评估结果证明了CoT提示方法在代码推理任务中的有效性。除CoT外，其他已被证实适用于自然语言推理任务的提示技术（如思维树[64]）同样可应用于代码推理任务；此外，专为程序执行推理设计的提示方法也值得深入研究。

致谢

本研究由宁波市自然科学基金（项目编号：2023J292）资助，并得到杭州城市大学超级计算中心提供的先进计算资源支持。

REFERENCES

- [1] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, “Recent advances in natural language processing via large pre-trained language models: A survey,” *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.
- [2] F. Barbieri, L. E. Anke, and J. Camacho-Collados, “Xlm-t: Multilingual language models in twitter for sentiment analysis and beyond,” in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2022, pp. 258–266.
- [3] A. Creswell, M. Shanahan, and I. Higgins, “Selection-inference: Exploiting large language models for interpretable logical reasoning,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [4] A. Rogers, M. Gardner, and I. Augenstein, “Qa dataset explosion: A taxonomy of nlp resources for question answering and reading comprehension,” *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–45, 2023.
- [5] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” *arXiv preprint arXiv:2310.03533*, 2023.
- [6] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [7] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, “Magicoder: Source code is all you need,” *arXiv preprint arXiv:2312.02120*, 2023.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [9] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [10] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.
- [11] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, “A survey on evaluation of large language models,” *ACM Transactions on Intelligent Systems and Technology*, 2023.
- [12] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. Wang, and X. V. Lin, “Lever: Learning to verify language-to-code generation with execution,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 106–26 128.
- [13] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” *arXiv preprint arXiv:2304.05128*, 2023.
- [14] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang, “Cruzeval: A benchmark for code reasoning, understanding and execution,” *arXiv preprint arXiv:2401.03065*, 2024.
- [15] C. Liu, S. D. Zhang, and R. Jabbarvand, “Codemind: A framework to challenge large language models for code reasoning,” *arXiv preprint arXiv:2402.09664*, 2024.
- [16] Y. Ding, B. Steenhoeck, K. Pei, G. Kaiser, W. Le, and B. Ray, “Traced: Execution-aware pre-training for source code,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [17] Y. Peng, C. Gao, Z. Li, B. Gao, D. Lo, Q. Zhang, and M. Lyu, “Static inference meets deep learning: a hybrid type inference approach for python,” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2019–2030.
- [18] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, “Graphcodebert: Pre-training code representations with data flow,” *arXiv preprint arXiv:2009.08366*, 2020.
- [19] “GitHub Copilot · Your AI pair programmer,” <https://github.com/features/copilot>, last accessed Mar. 2024.
- [20] M. Allamanis, S. Panthaplackel, and P. Yin, “Unsupervised evaluation of code llms with round-trip correctness,” *arXiv preprint arXiv:2402.08699*, 2024.
- [21] M. J. Min, Y. Ding, L. Buratti, S. Pujar, G. Kaiser, S. Jana, and B. Ray, “Beyond accuracy: Evaluating self-consistency of code llms,” in *The Twelfth International Conference on Learning Representations*, 2023.
- [22] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, “Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024.
- [23] J. Henkel, S. K. Lahiri, B. Liblit, and T. Repts, “Code vectors: understanding programs through embedded abstracted symbolic traces,” in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 163–174. [Online]. Available: <https://doi.org/10.1145/3236024.3236085>
- [24] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, and N. Duan, “Code execution with pre-trained language models,” *arXiv preprint arXiv:2305.05383*, 2023.
- [25] F. Tsimpouras, G. Rooijackers, A. Rajan, and M. Allamanis, “Embedding and classifying test execution traces using neural networks,” *IET Software*, vol. 16, no. 3, pp. 301–316, 2022. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/sfw2.12038>
- [26] E. Jabbar, H. Hemmati, and R. Feldt, “Investigating execution trace embedding for test case prioritization,” in *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, 2023, pp. 279–290.
- [27] E. La Malfa, C. Weinhuber, O. Torre, F. Lin, A. Cohn, N. Shadbolt, and M. Wooldridge, “Code simulation challenges for large language models,” *arXiv preprint arXiv:2401.09074*, 2024.
- [28] M. Jang, D. S. Kwon, and T. Lukasiewicz, “Becel: Benchmark for consistency evaluation of language models,” in *Proceedings of the 29th International Conference on Computational Linguistics*, 2022, pp. 3680–3696.
- [29] J. Chen, Z. Li, X. Hu, and X. Xia, “Nlperturbator: Studying the robustness of code llms to natural language variations,” *arXiv preprint arXiv:2406.19783*, 2024.
- [30] M. Jang and T. Lukasiewicz, “Consistency analysis of chatgpt,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 15 970–15 985.
- [31] Y. Elazar, N. Kassner, S. Ravfogel, A. Ravichander, E. Hovy, H. Schütze, and Y. Goldberg, “Measuring and improving consistency in pretrained language models,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 1012–1031, 2021.
- [32] P. Sahu, M. Cogswell, Y. Gong, and A. Divakaran, “Unpacking large language models with conceptual consistency,” *arXiv preprint arXiv:2209.15093*, 2022.
- [33] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [34] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [35] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi *et al.*, “Starcode 2 and the stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024.
- [36] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, “Codegen2: Lessons for training llms on programming and natural languages,” *arXiv preprint arXiv:2305.02309*, 2023.
- [37] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma, G. Liang, Y. Li, Q. Wang, and T. Xie, “Codereval: A benchmark of pragmatic code generation with generative pre-trained models,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [38] Y. Ding, Z. Wang, W. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth *et al.*, “Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [39] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang, and T. Yu, “Ds-1000: A natural and reliable benchmark for data science code generation,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 18 319–18 345.
- [40] F. Cassano, L. Li, A. Sethi, N. Shinn, A. Brennan-Jones, A. Lozhkov, C. Anderson, and A. Guha, “Can it edit? evaluating the ability of large language models to follow code editing instructions,” *arXiv preprint arXiv:2312.12450*, 2023.

参考文献

- [1] B. Min, H. Ross, E. Sulem, A. P. B. Veysch, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz and D. Roth, 《基于大型预训练语言模型的自然语言处理最新进展: 综述》, *ACM计算综述*, 第56卷第2期, 第1–40页, 2023年。
- [2] F. 巴尔贝里、L. E. 安克和J. 卡马乔-科拉多斯, 《Xlm-t: Twitter上的多语言语言模型在情感分析及其他领域的应用》, 载于《第十三届语言资源与评估会议论文集》, 2022年, 第258–266页。
- [3] A. Creswell, M. Shanahan and I. Higgins, 《选择推断: 利用大型语言模型进行可解释的逻辑推理》, 收录于《第十一届学习表征国际会议》, 2022年。
- [4] A. 罗杰斯、M. 加德纳和I. 奥根斯坦, 《问答数据集的爆发: 面向问答与阅读理解的自然语言处理资源分类体系》, 《ACM计算综述》, 第55卷第10期, 第1–45页, 2023年。
- [5] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo and J. M. Zhang, 《软件工程中的大型语言模型: 现状与开放性问题》, *arXiv预印本 arXiv:2310.03533*, 2023年。
- [6] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin 等, 《代码骆驼: 面向代码的开放基础模型》, *arXiv预印本 arXiv:2308.12950*, 2023年。
- [7] 魏宇、王泽、刘杰、丁勇和张磊, 《Magicoder: 源代码即是你所需的一切》, *arXiv预印本 arXiv:2312.02120*, 2023年。
- [8] 陈明、J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman 等人, 《评估基于代码训练的大语言模型》, *arXiv预印本 arXiv:2107.03374*, 2021年。
- [9] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le 等, “利用大型语言模型进行程序合成”, *arXiv预印本 arXiv:2108.07732*, 2021年。
- [10] 卢思、郭丹、任帅、黄杰、斯维亚特科夫斯基·A、布兰科·A、克莱门特·C、德雷恩·D、江丹、唐丹等, 《Codexglue: 一个用于代码理解与生成的机器学习基准数据集》, 收录于《第三十五届神经信息处理系统会议数据集与基准测试专题研讨会 (第一轮)》, 2021年。
- [11] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang 等, 《大型语言模型评估综述》, *ACM智能系统与技术汇刊*, 2023年。
- [12] A. 尼、S. 艾尔、D. 拉德夫、V. 斯托亚诺夫、W.-t. 叶、S. 王和X. V. 林, 《杠杆: 学习通过执行验证语言到代码生成》, 载于《国际机器学习会议》. PMLR, 2023年, 第26 106–26 128页。
- [13] 陈晓、林明、N·施äRLI和D. Zhou合著, 《训练大型语言模型进行自我调试》, *arXiv预印本 arXiv:2304.05128*, 2023年。
- [14] A. 古, B. 罗齐èRe, H. Leather, A. Solar-Lezama, G. Synnaeve 和 S. I. Wang, 《Cruxeval: 代码推理、理解与执行的基准》, *arXiv预印本 arXiv:2401.03065*, 2024年。
- [15] C. 刘、S. D. 张和 R. 贾巴尔万德, 《Codemind: 一种用于挑战大型语言模型代码推理能力的框架》, *arXiv预印本 arXiv:2402.09664*, 2024年。
- [16] 丁宇、B·斯滕霍克、裴科、G·凯泽、W·勒和B·雷, 《追踪: 面向源代码的执行感知预训练》, 载于《第46届IEEE/ACM软件工程国际会议论文集》, 2024年, 第1–12页。
- [17] 彭勇、高超、李志、高斌、罗丹、张强和吕明, “静态推理与深度学习的融合: 一种面向Python的混合型类型推断方法”, 载于《第44届国际软件工程会议论文集》, 2022年, 第2019–2030页。
- [18] 郭D、任S、卢S、冯Z、唐D、刘S、周L、段N、斯维亚特科夫斯基A、傅S等, “Graphcodebert: 基于数据流的预训练代码表示”, *arXiv预印本 arXiv:2009.08366*, 2020年。
- [19] “GitHub Copilot·您的人工智能配对程序员”, <https://github.com/features/copilot>, 2024年, 最后访问时间: 2024年3月。
- [20] M. Allamanis, S. Panthaplackel and P. Yin, 《基于往返正确性的代码LLM无监督评估》, *arXiv预印本 arXiv:2402.08699*, 2024年。
- [21] M. J. Min, Y. Ding, L. Buratti, S. Pujar, G. Kaiser, S. Jana 和 B. Ray, 《超越准确性: 评估代码式学习模型的自一致性》, 载于《第十二届国际学习表征会议》, 2023年。
- [22] X. 杜、M. 刘、K. 王、H. 王、J. 刘、Y. 陈、J. 冯、C. 沙、X. 彭和Y. 洛, 《Classeval: 一个用于评估类代码生成方面大型语言模型性能的手工构建基准》, 载于《第46届IEEE/ACM国际软件工程会议论文集》, 2024年。
- [23] J. Henkel, S. K. Lahiri, B. Liblit 与 T. Reps 合著, 《代码向量: 通过嵌入式抽象符号轨迹理解程序》, 收录于《2018年第26届ACM European Software Engineering Conference Joint Meeting 暨 Software Engineering 基础研讨会论文集》(ESEC / FSE 2018系列), 美国纽约州纽约市: 计算机协会, 2018年, 第163–174页。[在线版]。可获取地址: <https://doi.org/10.1145/3236024.3236085>
- [24] C. 刘、S. 卢、W. 陈、D. 江、A. 斯维亚特科夫斯基、S. 费、N. 孙达雷斯和 N. 杜安, 《使用预训练语言模型执行代码》, *arXiv预印本 arXiv:2305.05383*, 2023年。
- [25] F. Tsimpouras, G. Rooijackers, A. Rajan 和 M. Allamanis, 《利用神经网络嵌入与分类测试执行轨迹》, 《IET Software》, 第16卷第3期, 第301–316页, 2022年。[在线版]。可获取地址: <https://ie-tresearch.onlinelibrary.wiley.com/doi/abs/10.1049/sfw2.12038>
- [26] E. Jabbar, H. Hemmati 与 R. Feldt, 《研究执行轨迹嵌入在测试用例优先级排序中的应用》, 收录于《2023年IEEE第23届软件质量、可靠性与安全性国际会议 (QRS)》, 2023年, 第279–290页。
- [27] E. 拉·马尔法、C. 魏因胡贝尔、O. 托雷、F. 林、A. 科恩、N. 沙德博尔特和 M. 沃尔德里奇, 《大型语言模型的代码模拟挑战》, *arXiv预印本 arXiv:2401.09074*, 2024年。
- [28] 张明 (M. Jang)、权德善 (D. S. Kwon) 与卢卡谢维奇·T. (T. Lukasiewicz) 合著, 《Becel: 语言模型一致性评估的基准》, 载于《第29届国际计算语言学会议论文集》, 2022年, 第3680–3696页。
- [29] J. Chen, Z. Li, X. Hu 和 X. Xia, 《Nlperturbator: 研究代码语言模型对自然语言变异的鲁棒性》, *arXiv预印本 arXiv:2406.19783*, 2024年。
- [30] M. Jang 和 T. Lukasiewicz, 《ChatGPT的一致性分析》, 载于《2023年自然语言处理实证方法会议论文集》, 2023年, 第15 970–15 985页。
- [31] Y. Elazar, N. Kassner, S. Ravfogel, A. Ravichander, E. Hovy, H. Schütze 和 Y. Goldberg, 《测量与提升预训练语言模型的一致性》, 《计算语言学协会会刊》, 第9卷, 第1012–1031页, 2021年。
- [32] P. Sahu, M. Cogswell, Y. Gong 和 A. Divakaran, 《基于概念一致性的大型语言模型解构》, *arXiv预印本 arXiv:2209.15093*, 2022年。
- [33] 王旭、魏杰、舒尔曼斯·D、黎启华、池恩辉、纳朗·S、乔杜里·A和周丹, “自一致性可提升语言模型中的思维链推理能力”, 载于《第十一届学习表征国际会议》, 2022年。
- [34] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaci, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale 等, 《Llama 2: 开源基础模型与微调聊天模型》, *arXiv预印本 arXiv:2307.09288*, 2023年。
- [35] A. 洛日科夫、R. 李、L. B. 阿拉尔、F. 卡萨诺、J. 拉米-普瓦里耶、N. 塔齐等, 《Starcode 2与stack v2: 下一代工具》, *arXiv预印本 arXiv:2402.19173*, 2024年。
- [36] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese 和 Y. Zhou, 《Codegen2: 在编程和自然语言领域训练人工智能模型的经验教训》, *arXiv预印本 arXiv:2305.02309*, 2023年。
- [37] 余浩、沈斌、冉丹、张杰、张强、马勇、梁刚、李阳、王强和谢涛, 《Codereval: 基于生成式预训练模型的实用代码生成基准》, 载于《第46届IEEE/ACM软件工程国际会议论文集》, 2024年, 第1–12页。
- [38] 丁宇、王泽、艾哈迈德·W、丁浩、谭明、贾因·N、拉马纳坦·M·K、纳拉帕蒂·R、巴蒂亚·P、罗斯·D等, 《Crosscodeval: 一种多样化的多语言跨文件代码补全基准》, 《神经信息处理系统进展》, 第36卷, 2024年。
- [39] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang 和 T. Yu, 《Ds-1000: 数据科学代码生成的天然可靠基准》, 载于《国际机器学习会议PMLR》, 2023年, 第18 319–18 345页。
- [40] F. 卡萨诺、L. 李、A. 赛西、N. 辛恩、A. 布伦南-琼斯、A. 洛日科夫、C. 安德森和 A. 古哈, “它能编辑吗? 评估大型语言模型遵循代码编辑指令的能力”, *arXiv预印本 arXiv:2312.12450*, 2023年。

- [41] M. A. M. Khan, M. S. Bari, X. L. Do, W. Wang, M. R. Parvez, and S. Joty, "xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," *arXiv preprint arXiv:2303.03004*, 2023.
- [42] M. Tufano, S. Chandel, A. Agarwal, N. Sundaresan, and C. Clement, "Predicting code coverage without execution," *arXiv preprint arXiv:2307.13383*, 2023.
- [43] H. Hemmati, "How effective are code coverage criteria?" in *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 2015, pp. 151–156.
- [44] P. Ammann and J. Offutt, *Introduction to software testing*. Cambridge University Press, 2016.
- [45] B. Souza and M. Pradel, "Lexecutor: Learning-guided execution," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1522–1534.
- [46] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.
- [47] J. Li, G. Li, Y. Zhao, Y. Li, Z. Jin, H. Zhu, H. Liu, K. Liu, L. Wang, Z. Fang *et al.*, "Deveval: Evaluating code generation in practical software projects," *arXiv preprint arXiv:2401.06401*, 2024.
- [48] K. Zhang, Z. Li, J. Li, G. Li, and Z. Jin, "Self-edit: Fault-aware code editor for code generation," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 769–787. [Online]. Available: <https://aclanthology.org/2023.acl-long.45>
- [49] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *arXiv preprint arXiv:2308.10620*, 2023.
- [50] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [51] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.
- [52] A. Z. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large language models for test-free fault localization," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [53] A. G. Shypula, A. Madaan, Y. Zeng, U. Alon, J. R. Gardner, Y. Yang, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani, and A. Yazdanbakhsh, "Learning performance-improving code edits," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=ix7rLVHXyY>
- [54] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 172–184.
- [55] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [56] J. Chen, X. Hu, Z. Li, C. Gao, X. Xia, and D. Lo, "Code search is all you need? improving code suggestions with code search," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*, 2024, pp. 73:1–73:13.
- [57] Y. Song, T. Wang, P. Cai, S. K. Mondal, and J. P. Sahoo, "A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–40, 2023.
- [58] "Models - Hugging Face," <https://huggingface.co/models>, 2024, last accessed Mar. 2024.
- [59] "API Reference - OpenAI API," <https://platform.openai.com/docs/api-reference>, 2024, last accessed Mar. 2024.
- [60] "openai_humaneval · Datasets at Hugging Face," https://huggingface.co/datasets/openai_humaneval, 2024, last accessed Mar. 2024.
- [61] "FudanSELab/ClassEval · Datasets at Hugging Face," <https://huggingface.co/datasets/FudanSELab/ClassEval>, 2024, last accessed Mar. 2024.
- [62] "Replication package," <https://figshare.com/s/e5de95bd79ab5ddea76c>, 2024, replication package.
- [63] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [64] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

- [41] M. A. M. Khan, M. S. Bari, X. L. Do, W. Wang, M. R. Parvez 和 S. Joty, 《xcodeeval: 一个用于代码理解、生成、翻译和检索的大规模多语言多任务基准测试》, *arXiv 预印本 arXiv:2303.03004*, 2023年。
- [42] M. Tufano, S. Chandel, A. Agarwal, N. Sundaresan 和 C. Clement, 《无需执行即可预测代码覆盖率》, *arXiv 预印本 arXiv:2307.13383*, 2023年。
- [43] H. Hemmati, 《代码覆盖率标准的有效性如何?》, 收录于2015年IEEE软件质量、可靠性和安全性国际会议。IEEE, 2015年, 第151–156页。
- [44] P. 阿曼与 J. 奥菲特, 《软件测试导论》, 剑桥大学出版社, 2016年。
- [45] B. Souza 和 M. Pradel, 《执行器: 学习引导下的执行》, 载于第31届ACM欧洲联合软件工程会议暨软件工程基础研讨会论文集, 2023年, 第1522–1534页。
- [46] 陈明、J. 特沃雷克、H. 朱恩、Q. 袁、H. P. 德·奥利维拉·平托、J. 卡普兰、H. 爱德华兹、Y. 布尔达、N. 约瑟夫、G. 布罗克曼、A. 雷、R. 普里、G. 克鲁格、M. 彼得罗夫、H. 哈拉夫、G. 萨斯特里、P. 米什金、B. 陈、S. 格雷、N. 莱德、M. 帕夫洛夫、A. 波尔、L. 凯泽、M. 巴伐利亚、C. 温特、P. 提莱特、F. P. 苏赫、D. 康明斯、M. 普拉佩特、F. 查恩齐斯、E. 巴恩斯、A. 赫伯特·沃斯、W. H. 古斯、A. 尼科尔、A. 帕伊诺、N. 特扎克、J. 唐、I. 巴布什金、S. 巴拉吉、S. 贾因、W. 萨unders、C. 赫塞、A. N. 卡尔、J. 莱克、J. 阿恰姆、V. 米斯拉、E. 莫里卡瓦、A. 拉德福德、M. 奈特、M. 布伦戴奇、M. 穆拉蒂、K. 迈耶、P. 韦林德、B. 麦格鲁、D. 阿莫代伊、S. 麦坎德利什、I. 苏茨克韦尔以及 W. 扎伦巴, 《评估基于代码训练的大规模语言模型》, 2021年。
- [47] 李杰、李刚、赵宇、李勇、金哲、朱浩、刘浩、刘科、王磊、方志等人, 《Deveval: 在实际软件项目中评估代码生成》, *arXiv 预印本 arXiv:2401.06401*, 2024年。
- [48] K. Zhang, Z. Li, J. Li, G. Li 和 Z. Jin, 《自编辑: 面向代码生成的故障感知代码编辑器》, 载于《计算语言学协会第61届年会论文集(第一卷: 长篇论文)》(A. Rogers, J. Boyd-Graber 和 N. Okazaki 编), 加拿大多伦多: 计算语言学协会, 2023年7月, 第769–787页。[在线版]。可获取地址: <https://aclanthology.org/2023.acl-long.45>
- [49] X. 侯、Y. 赵、Y. 刘、Z. 杨、K. 王、L. 李、X. 罗、D. 洛、J. 格伦迪和 H. 王, “面向软件工程的大语言模型: 系统性文献综述”, *arXiv 预印本 arXiv:2308.10620*, 2023年。
- [50] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat 等, 《GPT-4技术报告》, *arXiv 预印本 arXiv:2303.08774*, 2023年。
- [51] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier 等, “Mistral 7b”, *arXiv 预印本 arXiv:2310.06825*, 2023年。
- [52] A. Z. 杨、C. 莱古斯、R. 马丁斯和 V. 黑伦多恩, 《无需测试的大型语言模型故障定位》, 载于第46届IEEE/ACM软件工程国际会议论文集, 2024年, 第1–12页。
- [53] A. G. Shypula, A. Madaan, Y. Zeng, U. Alon, J. R. Gardner, Y. Yang, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani 和 A. Yazdanbakhsh, “提升学习表现的代码修改方法”, 载于第十二届国际学习表征会议, 2024年[在线]。可获取地址: <https://openreview.net/forum?id=ix7rLVHXyY>
- [54] 魏勇、夏春生和张磊, 《协同驾驶辅助系统: 将大型语言模型与补全引擎融合用于自动化程序修复》, 载于第31届ACM欧洲联合软件工程会议暨软件工程基础研讨会论文集, 2023年, 第172–184页。
- [55] 魏杰、王旭、D·舒尔曼斯、M·博斯马、夏峰、E·奇、Q·V·勒、周丹等, 《思维链提示可激发大型语言模型的推理能力》, *神经信息处理系统进展*, 第35卷, 第24 824–24 837页, 2022年。
- [56] J. Chen, X. Hu, Z. Li, C. Gao, X. Xia 和 D. Lo, 《代码搜索就是全部? 利用代码搜索改进代码建议》, 载于第46届IEEE/ACM软件工程国际会议论文集 (ICSE 2024, 葡萄牙里斯本, 2024年4月14–20日), 第73卷: 第1–13页。
- [57] 宋宇、王涛、蔡鹏、S·K·蒙达尔和J·P·萨胡, 《小样本学习的全面综述: 演变历程、应用领域、挑战与机遇》, 《ACM计算综述》, 第55卷第13s期, 第1–40页, 2023年。
- [58] “模型 - Hugging Face”, <https://huggingface.co/models>, 2024年, 最后访问日期: 2024年3月。
- [59] “API 参考 - OpenAI API”, <https://platform.openai.com/docs/api-reference>, 2024年, 最后访问日期: 2024年3月。
- [60] OpenAI Humaneval · Hugging Face 数据集, <https://huggingface.co/datasets/openai-humaneval>, 2024年, 最后访问日期: 2024年3月。
- [61] “复旦SELab/ClassEval·Hugging Face数据集”, <https://huggingface.co/datasets/FudanSELab/ClassEval>, 2024年, 最后访问于2024年3月。
- [62] “复制包”, <https://figshare.com/s/e5de95bd79ab5ddea76c>, 2024年。
- [63] 吴奎 (W. Kwon)、李志 (Z. Li)、庄帅 (S. Zhuang)、盛宇 (Y. Sheng)、郑亮 (L. Zheng)、余春华 (C. H. Yu)、冈萨雷斯·J. E. (J. E. Gonzalez)、张浩 (H. Zhang) 与斯托伊卡 (I. Stoica) 合著《基于分页注意力机制的大规模语言模型高效内存管理》, 收录于ACM SIGOPS 第29届操作系统原理研讨会论文集, 2023年。
- [64] 姚思、余丹、赵杰、沙夫兰·I、格里菲斯·T、曹宇和纳拉辛汉·K, 《思维之树: 基于大型语言模型的刻意问题解决》, 《神经信息处理系统进展》, 第36卷, 2024年。